ECE 592 Project 2: Peer Grading, Side-Channel Attack Reproduction, and Evidence of Data-Memory-Dependent Prefetching

Group G
North Carolina State University
Abbott Sherwin, Manas Thumma, Mithoon Sabap, Siddarth Banakar

October 14, 2026

Abstract

This report is organized into three phases. Phase 1 (Section 2) peer-grades the Homework 1 baselines, distills five recurring measurement challenges, links the grading spreadsheet and the AI-usage analysis, evaluates three historical predictions (H1–H3), and analyzes SHiP++ overhead as the LLC grows. Phase 2 (Section 3) reproduces and compares Flush+Flush, Prime+Probe, and Flush+Reload across the Ajorpaz lab CPUs. Phase 3 (Section 4) presents a detection test for data-memory-dependent prefetching (DMP) and classifies each lab machine. Section 5 discusses relevance to stakeholders, records proposals/objections/evidence, and lists contributions.

1 Introduction

This report highlights three very important parts of reverse engineering microarchitecture. First is the review of academic work in this field. Reading and keeping up to date with the latest security breaches and findings in the reverse-engineering field is paramount to being able to avoid attacks and vulnerabilites. As such, the skill of being able to read, digest, and evaluate research work is critical. The first task of this homework was just that - reading over research work from a previous assignment and finding both the strengths and weaknesses of it. This allowed students to not only read other students' work, but also correlate it to other established works in the area, and provide critical analysis of both.

Second, the formation of three POCs - Flush+Flush, Flush+Reload, and Prime+Probe. This section helps formulate the basis of general-purpose attacks and how to think in terms of how a processor works in order to implement these attacks. This is the true experimentation part of the project, and focuses on the vulnerabilities that modern processors may have, and how to generate attacks to exploit them. It also gave the opportunity to compare and contrast these attacks, and see how one may be more viable than another given the circumstances. Finally, after developing and executing these attacks, a discussion is then had about the potential defenses we may have against attacks of this type.

Finally, the investigation part of this report focuses on establishing proof of a DMP (Data Memory-Dependent Prefetcher). By leveraging a targeted benchmark across a variety of machines, the results can illuminate the presence of such a construct. This part of the report helps build critical skills regarding the development of a basis of proof of the existence of different structures.

The report is then concluded by a discussion of the usefulness of the findings of the project in different fields and industries, as well as the implications of the results found across all experiments.

2 Peer Grading of Homework 1 and Analysis

2.1 Five Recurring Measurement Challenges in Homework 1

For each challenge we contrast a baseline that handled it poorly (*Left / Challenge*) with one that handled it well (*Right / Improvement*). The setup code for each side is reproduced beneath the analysis table, and the result graphs are shown as paired figures.

2.1.1 Challenge 1: Converting Timer Ticks into Real Time

Comparison	**Left (Challenge) – Baseline 12**	**Right (Improvement) – Baseline 10**
Block Role / Difference	ARM and x86 timers measured on each architecture independently, so there is no calibration between the two timing schemes.	`tsc_calibrate` is an example of timing-tick coordination (x86 only here). It converts ticks into ns, a method of standardization.
Observations	The entire graph is in ticks, but x86 and ARM ticks differ, which is why ARM is such a clear outlier.	Normalized to ns per access for both architectures. As a result, ARM is much more aligned with the other x86 machines.
Challenge	Not normalizing ticks across architectures (x86 vs. ARM) leads to measurements that misrepresent outliers.	
Root Cause	Each ISA can be timed with its own machine ticks, but ns-per-tick differs greatly. Comparing across platforms without converting to ns causes misrepresentation.	
Solution / Evidence	Convert ticks to ns for both platforms using the conversion rate from PMU counters or published metrics (right example).	
Remaining Limits	No major remaining limits; conversion to ns handles timing differences. Collecting ticks is a standardized way to view data, and conversion normalizes it.	

Table 1: Challenge 1 – timer-tick normalization across ISAs.

Left (Baseline 12) – setup block:

```
static inline uint64_t arm_counter_start(void) {
    uint64_t t;
    __asm__ __volatile__(
        "dsb sy\n\t"
        "isb\n\t"
        "mrs %0, cntvct_el0\n\t"
        "isb\n\t"
        : "=r"(t) :: "memory");
    return t;
}
static inline uint64_t x86_tsc_start(void) {
    _mm_lfence();
    uint64_t t = __rdtsc();
    _mm_lfence();
    return t;
```

```
}
static inline uint64_t x86_tsc_stop(void) {
    unsigned aux;
    uint64_t t = __rdtscp(&aux);
    _mm_lfence();
    return t;
}
```

Right (Baseline 10) – setup block:

```
static double now_seconds(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}
static int cmp_double(const void *a, const void *b) {
    double da = *(const double *)a, db = *(const double *)b;
    return (da > db) - (da < db);
}
```

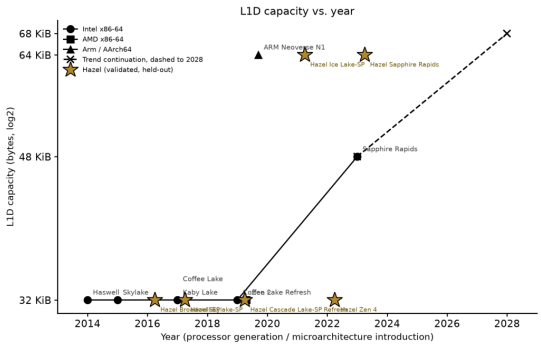
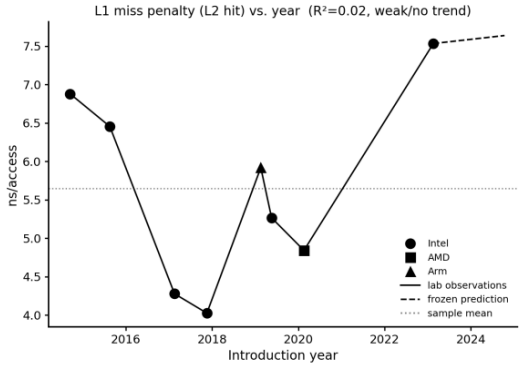


Figure 127: L1D capacity vs. year.



(b) L1 miss penalty. $R^2 = 0.02$.

Figure 1: Challenge 1. Left (Baseline 12): latency in raw ticks, ARM is an outlier. Right (Baseline 10): ns-per-access normalization aligns ARM with x86.

2.1.2 Challenge 2: Last-Level Cache (LLC) Capacity Investigation

Comparison	Left (Challenge) – T14	Right (Improvement) – T09
Block Role / Difference	For LLC capacity (and all levels) this code used mainly power-of-two investigation, using $\log_2$ and more coarse sweeps.	This code does not assume power-of-two granularity for sweeps and uses `pts_per_octave` to control how densely boundaries are sampled.
Observations	Unclear transition boundaries between L2/LLC and earlier. Even with randomized access, there is no clean boundary, suggesting the LLC boundary is not on a power-of-two point.	Clean distinction between each level. The coarse sweep found an L2/LLC boundary on a clean non-power-of-two number (on a machine not targeted in P1, but results still hold).
Challenge	Power-of-two granularity works up to the LLC, where (due to large size change and design) LLC size is not constrained to a power of two.	
Root Cause	Modern processors need not constrain the LLC to a power of two because of per-core slicing. Power-of-two slices sum, by core count, to a non-power-of-two total.	
Solution / Evidence	Sweep more liberally across non-power-of-two capacities; use slicing intuition plus per-generation core count (available pre-PMU) to estimate LLC capacity off the power-of-two grid.	
Remaining Limits	Check how far LLC slicing goes and how it correlates with cache size; test whether a core's own partition is itself a power of two.	

Table 2: Challenge 2 – inferring LLC capacity.

Left (T14) – setup block:

```
static void exp_capacity(int argc, char **argv, FILE *csv) {
    long min_kb   = arg_get_long(argc, argv, "--min_kb", 4);
    long max_kb   = arg_get_long(argc, argv, "--max_kb", 131072);
    long ppo      = arg_get_long(argc, argv, "--points_per_octave", 4);
    long batch    = arg_get_long(argc, argv, "--batch", 10000);
    long reps     = arg_get_long(argc, argv, "--reps", 1000000);
    long stride_b = arg_get_long(argc, argv, "--stride_bytes", 256);
    uint32_t seed = (uint32_t)arg_get_long(argc, argv, "--seed", 12345);
    double log2_min = log2((double)min_kb), log2_max = log2((double)max_kb);
    long n_octaves  = (long)ceil(log2_max - log2_min);
    long n_points   = n_octaves * ppo + 1;
    for (long p = 0; p <= n_points; p++) {
        double frac = (double)p / (double)n_points;
        double kb   = pow(2.0, log2_min + frac * (log2_max - log2_min));
        size_t footprint_bytes = (size_t)(kb * 1024.0);
        size_t n = footprint_bytes / sizeof(struct node);
        if (n < 2) n = 2;
        /* ... */
    }
}
```

Right (T09) – setup block:

```
static void mode_capacity(cfg_t *cfg, size_t min_bytes, size_t max_bytes,
                          int pts_per_octave, size_t spacing) {
    int gh;
```

```
    unsigned char *r = alloc_region(max_bytes, cfg->pages_policy, &gh);
    /* Geometric sweep; pts_per_octave controls how densely boundaries
     * are sampled. The driver script reruns dense sweeps around each
     * transition. */
    double lo = (double)min_bytes, hi = (double)max_bytes;
    double step = pow(2.0, 1.0 / (double)pts_per_octave);
    size_t prev_n = 0;
    for (double w = lo; w <= hi * 1.0000001; w *= step) {
        size_t bytes = (size_t)w;
        size_t n = bytes / spacing;
        if (n < 4) continue;
        if (n == prev_n) continue; /* skip duplicate footprints */
        prev_n = n;
        /* ... */
    }
}
```

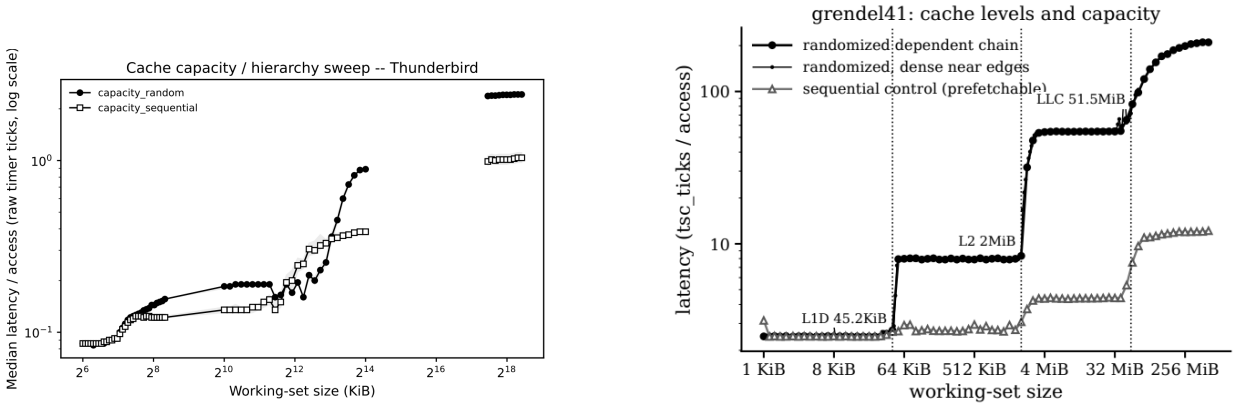


Figure 2: Challenge 2. Left (T14): coarse power-of-two sweep with unclear L2/LLC boundary. Right (T09): dense geometric sweep resolving a clean non-power-of-two boundary.

2.1.3 Challenge 3: Inclusion / Exclusion Results

Comparison	Left (Challenge) – Baseline 16	Right (Improvement) – Baseline 5
Block Role / Difference	Counted how many upper-level lines were evicted after filling a lower set, compared against a fixed way count. No specific target, timing threshold, or control, so observed misses cannot be isolated to the upper level.	A second core walks an eviction set for the target's LLC set while the first core times a single reload. Because the second core never touches the target, any miss can only come from back-invalidation, and a decoy set gives a control.
Observations	Little detail. Every host is reported 8/8 for L1–L2 and 16/16 for L2–LLC, including Artemisia and Thunderbird (12-way and 4-way L1). Fixed 8/16 values do not account for machine differences.	Clear results across machines. Most show over 98% of reloads missed when the core walked the eviction set; against a baseline this gave 0% reload miss. Artemisia is shown non-inclusive, matching its different configuration.
Challenge	The target was removed from the lower level without being evicted from the upper level. Baseline 16 cannot test for this and reports every host inclusive, even though Artemisia and Skylark are non-inclusive (partially shown by Baseline 5).	
Root Cause	The same thread filled the lower and upper level, and the test compares against a fixed 8/16 instead of measuring associativity. No baseline to validate against.	Similar issue: every condition read values the same and counted everything inclusive.
Solution / Evidence	Needs a mechanism (as in Baseline 5) so misses can only be explained by an upper-level event, plus a baseline for comparison.	The eviction set is found by search, and a decoy offset is probed to confirm no collision. The threshold moved to 25% between the hit and miss anchors, and verdicts use miss rates instead of medians. Artemisia is shown non-inclusive.
Remaining Limits	Data cannot be validated; a baseline run is needed for accurate comparison.	Skylark data was incomplete, so only Artemisia was proved non-inclusive.

Table 3: Challenge 3 – inclusion/exclusion classification.

Left (Baseline 16) – setup block:

```
// inclusion: fill a lower level's set, then probe the upper level
fill_set(L2_or_LLC, all_ways)
for way in range(upper_level_way_count):
    touch(upper_level_line[way])          // repopulate L1 (or L2)
probe_result = count(upper_level_line[way] evicted for way in range(...))
record(pair, upper_collisions=probe_result, n=upper_level_way_count)
```

Right (Baseline 5) – setup block:

```
helper_set = (c == 1) ? decoy : cand; ...
atomic_store(&helper_done, 0);
atomic_store(&helper_go, 1);
while (!atomic_load(&helper_done)) { }
if (c == 3) {
    uint64_t acc = 0;
    for (size_t off = 0; off < cfg.flush_bytes; off += 4096)
        acc += touch(flush + off);
    __asm__ __volatile__("" :: "r"(acc) : "memory");
    /* must measure a miss, not a miss plus a page walk,
     * or it is not comparable with the other three. */
```

```
    warm_translation(pool, cfg.offset);
}
uint64_t t0 = timer_start();
uint64_t v  = touch(target);
uint64_t t1 = timer_stop();
__asm__ __volatile__("" :: "r"(v) : "memory");
```

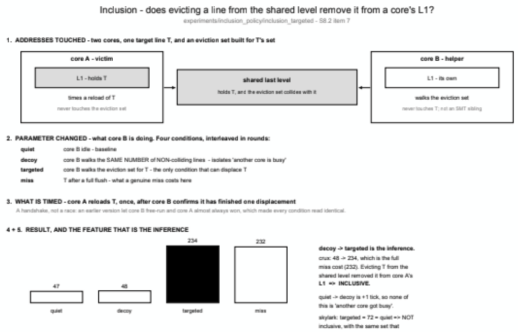


Figure 6: Inclusion: how a helper core evicting a line from the shared level tests whether the victim's private copy survives.

Host	L1 vs L2 collisions	L2 vs LLC collisions
sunbird	8/8	16/16
charnwood	8/8	16/16
ookay	8/8	16/16
upgrade	8/8	16/16
crux	8/8	16/16
gravedigger	8/8	16/16
sorrow	8/8	16/16
thunderbird	8/8	16/16

52

Host	L1 vs L2 collisions	L2 vs LLC collisions
skylark	8/8	16/16
elevation	8/8	16/16
artemisia	8/8	16/16
haswell	N/A	N/A
broadwell	N/A	N/A
skylake	8/8	16/16
cascadelake	8/8	16/16
icelake_6326	8/8	16/16
icelake_8358	8/8	16/16
sapphirerapids	8/8	16/16
genoa	8/8	16/16
turin	8/8	16/16

Machine	targeted − decoy (L3 misses/access)	target-half batches with an L3 miss	decoy-half batches with an L3 miss
Upgrade	0.8103	99.44%	0.00%
Crux	0.8139	99.89%	0.00%
Ookay	0.8109	99.69%	0.00%
Charnwood	0.8051	98.58%	0.00%
Sunbird	0.8179	99.79%	0.00%

Artemisia resolves NOT INCLUSIVE, and required a fix to the scrub. Every earlier attempt on this host refused at private-scrub qualification rather than at the anchors or the counters. The banked scrub built 32 nodes at a 64 KiB stride; that stride spans two L2 sets, so each set received about 16 lines, exactly the associativity of this part's 2 MiB L2, and qualification turned on replacement state rather than capacity. The 2026-09-16 reproduction shows the signature plainly: seven of eight targets passed every gate and target 2 alone failed, an aggregate L2-positive of 87.5% against a 90% gate, which is exactly 7/8. Doubling the scrub to 64 nodes (and growing the pool to hold it) qualified all eight targets. The resulting verdict is unambiguous:

Condition	Samples	L3-miss rate	Role
quiet	1,000,000	0.00%	baseline
decoy	1,000,000	0.00%	non-colliding control
targeted	1,000,000	0.00%	LLC set evicted, line in private L2, *survives*
cold	1,000,000	100.00%	positive control (`CLFLUSH`)
llc_only	1,000,000	0.00%	line is genuinely LLC-resident
llc_targeted	1,000,000	100.00%	the same eviction set *does* evict from the LLC

Figure 3: Challenge 3. Left (Baseline 16) and right (Baseline 5) inclusion/exclusion results.

2.1.4 Challenge 4: Finding the Cause of Slow Memory Access

Comparison	Left (Challenge) – Baseline 8	Right (Improvement) – Baseline 7
Shared blocks	Both tests follow a chain of memory addresses; each read tells the test where to read next.	
Block Role / Difference	Aligns the memory and touches it before testing. It does not check how the physical pages are arranged.	Checks three setups: ordinary 4 KiB pages, memory placed together but using 4 KiB translations, and large 2 MiB pages. It verifies the page setup.
Observations	The report warns that address-translation delays can look like a cache boundary, and checks this by changing the spacing between nodes.	With memory placed together, L2 refills fell from 0.1066 to 0.0001 per read, while translation refills stayed near 0.75 per read. Large pages reduced translation refills to almost zero.
Challenge	A slower read does not always mean the cache is full. Page placement and the work needed to find a physical address can also add delay.	
Root Cause	L2 refills fell while translation refills stayed similar, pointing to page placement or cache conflicts as the main cause of the first slowdown. The large-page result also shows an added translation cost.	
Solution / Evidence	Keep the test the same while changing the page setup; check the pages used, read times, and cache and translation counters. Baseline 7 uses this approach to help separate the causes.	
Remaining Limits	The numbers come from Baseline 7's report; its raw files were unavailable locally, so they do not prove Baseline 8 had the same problem, and the exact physical addresses were not measured.	

Table 4: Challenge 4 – separating cache capacity from address-translation and page-placement effects.

Left (Baseline 8) – setup block (selected lines, comments added):

```
page_size = sysconf(_SC_PAGESIZE);   // page_size: bytes per system page.
allocation_error = posix_memalign(&allocation, (size_t)page_size,
                                  allocation_bytes);
    // allocation: memory pointer; allocation_error: status;
    // allocation_bytes: requested bytes.
memset(allocation, 0, allocation_bytes); // Touch the memory before testing.
```

Right (Baseline 7) – setup block (selected lines, comments added):

```
int huge = strcmp(argv[1], "base") != 0; // huge: true when the mode (argv[1]) is not base.
long backing = huge_kb(base);            // backing: large-page KiB at pointer base.
if (backing != (huge ? REGION / 1024 : 0)) { // REGION: allocation bytes; check page backing.
    fprintf(stderr, "backing check failed: requested=%s AnonHugePages=%ld\n",
            argv[1], backing);           // Report the mode and page backing.
    return 1;                            // Stop if the page backing is wrong.
}                                        // End the check.
```

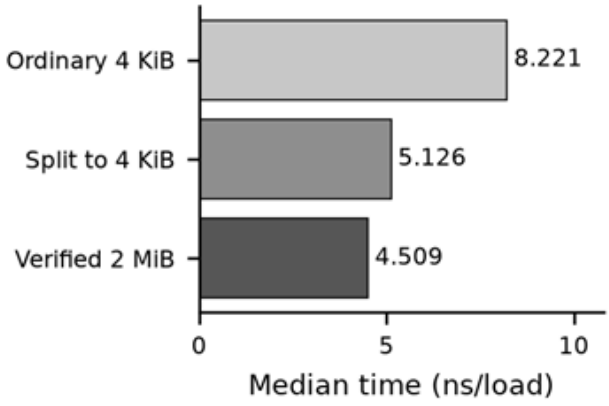


Figure 4: Challenge 4. Right (Baseline 7): Thunderbird, 768 KiB. "Split" means memory placed together with 4 KiB translations; each setup has 63 timed batches. Baseline 8's reviewed material did not show the same three-way page test.

2.1.5 Challenge 5: Associativity Investigations

Comparison	Left (Challenge) – Baseline 16	Right (Improvement) – Baseline 4
Block Role / Difference	Measured the *probability* that a line was evicted rather than the *access time*. Eviction probability should hit 1 when the stride exceeds associativity, but their data were too noisy to extract it.	Directly measured access latency for varying strides, looking for a sharp jump when the stride exceeds the associativity of the cache level.
Observations	Data too noisy to find associativity; many small steps in probability before the large jump to 1.	Clean transitions in eviction time per working-set size, allowing the associativity of each level to be read directly.
Challenge	Noise made it hard to extract associativity of higher levels; stairsteps at unpredictable states prevented a correct read.	
Root Cause	The probability metric is less robust than direct timing. It introduces evictions at $\frac{1}{2}$ or $\frac{1}{4}$ associativity and does not cleanly show a transition at a measured W.	
Solution / Evidence	Switch to direct timing and run fewer sweep sizes, each matched to a computed cache size (after a capacity analysis fixes L1/L2/LLC).	
Remaining	The Baseline 4 timing analysis still showed noise near multiples of associativity, but a theoretical framework let them identify the "real" transition.	

Table 5: Challenge 5 – inferring associativity.

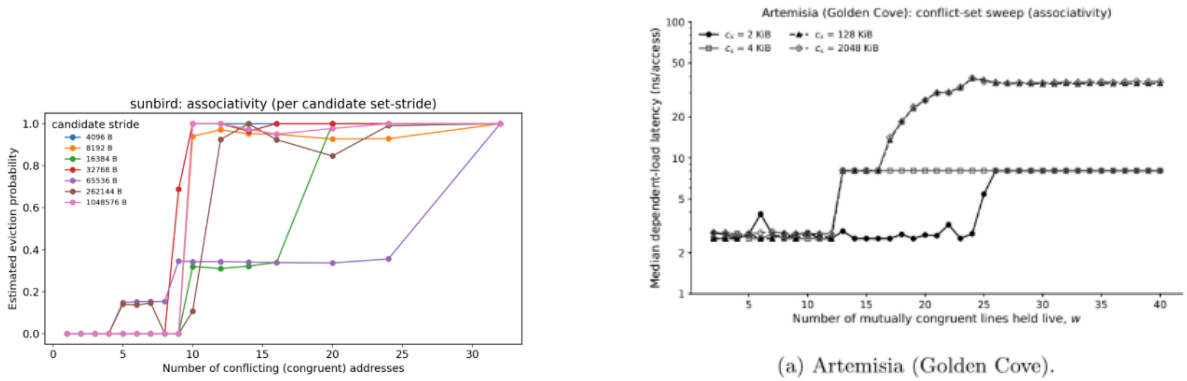
Left (Baseline 16) – setup block:

```
for S in CANDIDATE_STRIDES:
    for N in [1,2,3,4,5,6,7,8,9,10,12,14,16,20,24,32]:
        touch(marked_address)
        touch(N congruent_addresses, stride=S)   // attempt to evict
        evicted = probe(marked_address)          // miss => evicted
```

```
        record(S, N, eviction_probability += evicted)
```

Right (Baseline 4) – setup block:

```
size_t cstrides[] = {1*KiB,2*KiB,4*KiB,8*KiB,16*KiB,32*KiB,64*KiB,
                     128*KiB,256*KiB,512*KiB,1*MiB,2*MiB};
region_alloc(&reg, cstrides[ncs-1] * (max_ways + 2)); /* MADV_HUGEPAGE */
for (size_t ci = 0; ci < ncs; ci++) {
    size_t cs = cstrides[ci];
    for (int w = 2; w <= max_ways; w++) {
        for (int i = 0; i < w; i++)
            addr[i] = reg.base + (size_t)i * cs;  /* A_i = base + i*c_s */
        build_cycle_addrs(addr, w, c->seed, 1);   /* randomized cycle */
        measure_chain(addr[0], c->batch, c->reps, c->batch*4, v);
        compute_stats(v, c->reps, &s);
        emit_row(o, "assoc", "conflicting_lines", w,
                 "congruence_stride_bytes", cs, "random",
                 c->batch * c->reps, &s);
    }
}
```



(a) Artemisia (Golden Cove).

Figure 5: Challenge 5. Left (Baseline 16): noisy eviction-probability steps. Right (Baseline 4): clean timing transitions per working-set size.

2.2 Peer Grading Spreadsheet and AI-Usage Statement

The full peer-grading tables are maintained in a shared spreadsheet rather than reproduced here: Group G HW2 – Peer Grading and Championship Votes.

AI-Usage Analysis

Through reading the 17 baselines of Homework 1, AI seemed to have a diverse role in helping students accomplish different tasks. The main usage seems to be report and presentation formatting/writing, as well as data analysis and visualization. Another big area of AI usage was the automation of running data collection and experiments. Homework 1 had a large number of experiments to run with many different trials, so an automated AI agent could work behind the scenes to run code while students analyzed data and made predictions. Overall, some instances of code generation for the main benchmarks and analysis generation by AI were observed, but these were fewer in between and largely in the reports with less complete results or missing points.

This leads to the conclusion that while AI use was pervasive throughout all the baselines in HW01, the most successful teams (the highest scoring) tend to use it for helper tasks – formatting, automation, and visualization – rather than the bulk of the analysis and code-generation tasks. We

think this is because when AI does the thinking, or is expected to do all of it, the initial input into an LLM is not high quality: these large tasks are probably given as "do this completely," rather than telling the AI what is already understood, what gaps exist, and what else needs to be done. As a result, the AI gives broader, less useful analysis.

On the contrary, when AI is given a limited scope, it can lead to more polished reports, functional coverage of all requirements, and analysis that comes from a basis of knowledge. This is why many assignments that list AI use for code generation and analysis synthesis have good explanations in some sections but are often missing items, while baselines that use AI for helper tasks show more widespread analysis, with AI used to cover gaps rather than blanket the entire assignment.

2.3 Historical Predictions Evaluated Against Homework 1 (H1–H3)

2.3.1 H1 – Contradicted: Mark Hill, *A Case for Direct-Mapped Caches* (IEEE Computer, 1988)

Hill argued that direct-mapped caches would become the choice of cache organization because a fast hit can make up for extra misses. A direct map would be shorter to hit on than fully or set-associative caches, and he predicted the miss gap would shrink as caches got larger. The argument states that associativity added too much time and would never be overcome compared to direct-mapped, and that miss penalties would be small enough not to become a huge issue.

We found through our research into the baselines that the opposite actually proved true. Every team found that L1, L2, and LLC were at least somewhat associative, often with 4+ ways in each level; no cache was direct-mapped. Every team that reported L1 associativity found it set-associative, measuring 8-way on Haswell, Skylake, and Zen 2, 12-way on Sapphire Rapids, and 16-way on the Ampere machine, as shown in T14's chronological table and T09's per-machine table. Hill argued that as caches grow, the associativity increase would not meaningfully improve hit time and would cost the direct-hit-time benefit. However, in the supported laws of Homework 1, L1 hit latency is flat across the decade at roughly 2.64 ± 0.76 ns (the Thumma-Su "Latency Law") even as associativity held at 8-way.

This leads us to read Hill's prediction as contradicted by the baselines. It should be mentioned that Hill's paper is conditional, since it focuses on the benefits of a direct-mapped cache at one level and concedes that effectiveness may decrease with larger hierarchies. Still, even for the first level, the HW1 data is strong enough on associativity and hit latency to provide this contradiction.

2.3.2 H2 – Supported: Sally A. McKee, *Reflections on the Memory Wall* (Computing Frontiers, 2004)

The memory wall is a well-known boundary presented as a limitation of CPUs: the limits of memory hierarchy, size, and latency would all limit the effectiveness of CPU improvements, and largely they have. Memory access, and computation that uses memory, has repeatedly proved to be a limiting factor for large performance gains. However, the memory itself is often treated as the main point of contention; McKee provides a different perspective, showing the memory wall is true but entirely application dependent, and that for many processes the wall had not been hit yet (at the time of publication). The HW1 baselines show this in action through their randomized and pointer-chase accesses.

To elaborate, T14 reports that sequential-traversal latency stays below the randomized-traversal latency at larger footprints, the same pattern on every machine tested. T09 shows the same thing: the sequential chain tracks the randomized chain but diverges past the LLC. On T09's Sapphire Rapids machine the randomized chain climbs through plateaus of 2.47 ticks/access in L1, 7.89 in

L2, and 54.46 in the LLC, then reaches a DRAM level of 199.9 ticks/access, which at the measured 0.5 ns/tick is about 100 ns. T14's Sunbird numbers show the same gap, with an L1-class baseline near 10 ticks, an LLC-class plateau of 50 to 52 ticks, and a DRAM class around 220 ticks for the randomized chain, while the sequential control holds near the low end.

Therefore H2 is concluded as supported, with a caveat about prefetching. HW1 was a latency study, so it measured access sensitivity and latency across benchmark configurations, as McKee predicted with workloads. It did not measure sustained bandwidth, so the bandwidth-vs-latency part of her argument is left unresolved by our evidence. However, the sequential-versus-random results line up with her refinement that there should not be a single workload considered for evaluating where the memory wall lies.

2.3.3 H3 – Benefit May Not Persist Across Changing Workloads: Norman P. Jouppi, *Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers* (ISCA, 1990)

Jouppi proposed stream buffers that fetch successive cache lines ahead of demand. His exact claim: "Together, victim caches and stream buffers reduce the miss rate of the first level in the cache hierarchy by a factor of two to three on a set of six large benchmarks." The result was measured for a direct-mapped, 4 KB instruction and data first-level cache with 16-byte lines in a 1990 WRL system. The stream-buffer benefit depends on sufficiently sequential accesses, useful predictions, and enough memory bandwidth to deliver prefetched lines before they are needed.

HW1 illustrates why that benefit cannot be assumed to persist as workloads become more irregular. Baseline 14's sequential controls remain faster at large footprints on every machine tested, while randomized pointer chasing exposes substantially higher latency. This suggests a solution effective for sequential streams may lose value when future workloads depend on unpredictable addresses. Evidence: Baseline 14 report, Section 4.1 p. 24 and Figs. 2–9, pp. 25–32.

2.4 Task 1.3: SHiP++ as the LLC Gets Bigger

A bigger LLC would need more storage for SHiP++ in total. However, that storage can become a smaller share of the cache if we keep the predictor table and the number of sampled sets the same. The answer depends on which parts we choose to grow.

The LLC is the last-level cache, which holds data before the CPU has to fetch it from main memory. A replacement policy decides which cache line to remove when space is needed. SHiP++ learns from earlier accesses to help decide which data is worth keeping; its extra tracking information is called metadata.

What we assume in this example

We keep one CPU core, 64-byte cache lines, and 16 ways (slots per set). We make the cache bigger by adding more sets, not more ways. We keep 64 sampled sets and a predictor table with 16,384 entries. Prefetching is enabled, so we count the flag for data brought in before it is requested. These are example design choices, not cache settings measured in Homework 1.

Where the extra information is stored

We group the storage into three parts, using the costs in reference [38]: SHiP++ paper, section 4.

Where it is stored	What it keeps	What happens when we add more sets
On every cache line	A 2-bit replacement score and a 1-bit prefetch flag.	More lines need more bits. This part grows at the same rate as cache capacity.
Only in the 64 sampled sets	Each sampled line has a 14-bit request label and a 1-bit reuse flag; a list identifies the sampled sets.	Stays about 2 KiB here because the number of sampled sets and ways is fixed.
In the predictor table	16,384 small 3-bit counters that learn from the sampled sets; lines used by this core share this table.	Stays 6 KiB because the entry count is kept fixed.

Table 6: The three parts of SHiP++ storage.

The sampled sets let the policy learn from a small part of the cache. In this example they contain $64 \times 16 = 1{,}024$ lines. Their 15 bits per line take 1,920 bytes, and the list of 64 two-byte set IDs adds 128 bytes, giving 2 KiB total. The predictor and sampled information together take 8 KiB. With more cores the published design uses a predictor table per core and records which core brought in a sampled line; we hold the core count fixed. Normal and prefetched accesses share parts of the same predictor, so we do not count two full tables: SHiP++ code.

How much storage is needed

With these choices, total policy storage is $3 \times (\text{cache capacity in bytes})/512 + 8{,}192$ bytes. The first part is three bits per 64-byte line; the second is the fixed 8 KiB for learning.

LLC data capacity	Total SHiP++ policy storage	Share of LLC data capacity
2 MiB	20 KiB	0.977%
8 MiB	56 KiB	0.684%
32 MiB	200 KiB	0.610%

Table 7: Example storage as the LLC grows.

The 2 MiB row matches the paper's single-core storage budget; the larger rows are calculations using the same choices, not new measurements. The total bytes rise, but the share of the cache falls, approaching about 0.586% rather than zero because every line still needs three bits. These figures count all policy metadata in this example; they do not mean all of it is new (RRIP already uses the two replacement bits per line), and they leave out data, address tags, and the logic to read and update metadata.

What changes if we grow other parts

If we add ways instead of sets, each sampled set has more lines to track, so its metadata grows, and the policy has more candidate lines to consider. If we keep sampling the same percentage of sets, the number of sampled sets must grow with the cache, and enlarging the predictor adds storage; in either case the overhead percentage may fall less or stay roughly the same. Keeping just 64 sampled sets makes them a smaller part of a larger cache, which for the same number of accesses can mean fewer chances to train the predictor. A fixed predictor may also struggle if the program uses many more distinct access patterns. Cache size alone does not prove prediction will get worse. If set IDs need more bits, their storage grows too; two-byte IDs suffice for the sizes in our table.

Work, delay and energy

SHiP++ does not scan the whole cache on every access. It updates the line's replacement information, checks the predictor when inserting new data, and learns from hits and removals in sampled sets. Adding sets does not by itself require more of these steps per access; total update work still depends on how many accesses and misses the workload creates. Adding ways can increase the work to choose and age candidate lines (original SHiP++ code). More stored bits generally need more area and energy, and reading/updating them uses energy; larger tables, longer wires, or more ways can add delay. Some work may overlap other cache operations, so more storage does not automatically mean a longer hit delay, but we cannot assume the delay or energy cost stays fixed either.

When the benefit could be worth the cost

An application's footprint is the amount of data it uses. If that stays fixed and its useful data eventually fits comfortably in a larger cache, there may be fewer avoidable misses for SHiP++ to fix, and its benefit could shrink even though it still needs tracking bits and update work. If the useful data grows with the cache, or several programs compete for space, good replacement choices may still matter: SHiP++ could keep data that will be reused and give less space to data used once, and avoiding enough memory accesses could save more time and energy than the policy costs. But a large footprint alone is not enough; keeping data longer cannot help if it is never reused.

What our evidence cannot tell us

Homework 1 did not compare SHiP++ with another policy, nor measure its prediction accuracy, chip area, added delay, or energy use. Our cache-size sweeps did not establish exact LLC set and way counts, so the growth model above remains an assumption. The timings explain why avoiding some memory accesses could be useful, but they cannot tell us the actual speedup, how many misses SHiP++ would prevent, whether it would save energy overall, or the best predictor and sample sizes; those claims would need a direct policy comparison. No new experiments were run for this discussion. Evidence: Vinson Young, Chia-Chen Chou, Aamer Jaleel, and Moinuddin K. Qureshi, "SHiP++: Enhancing Signature-Based Hit Predictor for Improved Cache Performance," Second Cache Replacement Championship, 2017.

3 Attack Proof-of-Concept Reproduction and Comparison

3.1 Flush+Flush

Flush+Flush was implemented in two ways, single-thread and multi-thread. In the single-thread version, a simple proof-of-concept shows that an attacker could learn the cached and uncached flush time of a secret and use that to determine the presence of the secret. First, a sequence of warm-up caches and flushes was repeated, anywhere from 1,000 to 10,000 times, to warm up the machine into a stable state. Then another 10,000 training samples were run with `rdtsc()` timing each flush. A training function found the median of each list (cached vs. not) to set a threshold above or below which signifies cached or not. Then 10,000 more samples with results unknown to the process (but known overall) were timed, the process guessed from the training, and accuracy was computed against the truth.

The multi-thread model runs the same process, except the attacker (flush and timing) and victim (caching) are separated using the pthread library, with barriers to synchronize the threads

and depict a more realistic scenario.

Machine	Source	WU	N	Tr%	Te%	Med c	Med u	Thr	FP	FN	FPR%	MI
sunbird	output_ff	1k	10000	100.0	100.0	180	288	234	0	1	0.00	0.999
charnwood	output_ff	1k	10000	99.8	99.9	150	136	143	7	0	0.14	0.992
ookay	output_ff	1k	10000	94.4	95.5	164	154	159	395	57	7.90	0.752
upgrade	output_ff	1k	10000	77.2	97.0	139	129	134	54	250	1.08	0.813
crux	output_ff	1k	10000	99.8	93.5	136	114	125	4	642	0.08	0.707
artemisia	output_ff	1k	10000	100.0	99.4	262	390	326	9	47	0.18	0.952
greenforest	output_ff_cal	11k	10000	87.9	63.2	139	104	121	3	3677	0.06	0.144
elevation	output_ff_good	11k	50000	91.1	100.0	137	97	117	5	0	0.02	0.999

Table 8: Flush+Flush, single process. WU = warm-up; Med c/u = median cached/uncached ticks; Thr = threshold; MI = leakage mutual information (bits/probe). FN rates: greenforest 73.54%, crux 12.84%, others small. Greenforest and elevation re-run with +10k warm-up.

Machine	Source	WU	N	Tr%	Te%	Med c	Med u	Thr	FP	FN	FPR%	MI
sunbird	output_ff_multi	1k	10000	78.3	72.9	172	280	226	2666	42	54.99	0.248
charnwood	output_ff_multi	1k	10000	64.2	49.1	152	146	149	577	4515	11.90	0.000
ookay	output_ff_multi_low_cal	1k	10000	61.5	48.4	174	170	172	36	5123	0.74	0.000
upgrade	output_ff_multi	1k	10000	50.6	48.4	161	161	161	5	5151	0.10	0.000
crux	output_ff_multi	1k	10000	49.9	51.5	153	153	153	4848	1	100.00	0.000
artemisia	output_ff_multi	1k	10000	68.6	60.1	246	258	252	1258	2733	25.95	0.035
greenforest	output_ff_multi_cal	11k	10000	67.4	80.7	112	108	110	1933	0	38.75	0.408
elevation	output_ff_multi	1k	10000	78.3	77.1	200	152	176	2276	10	46.95	0.330

Table 9: Flush+Flush, multi-process (attacker/victim threads). Columns as in Table 8.

3.2 Prime+Probe

What we tried. We wanted to leak a single secret bit from a simulated victim. That bit determined which of the two L1 data-cache sets the victim accessed. The side channel was memory-access timing: after filling both sets with our own cache lines, we measured how long it took to read them again, where slower reads indicated the victim had evicted our lines. The microarchitectural detail was set-associative cache contention – different addresses map to the same set and compete for its limited ways – which let us infer the victim's selected set and therefore its secret bit within one process.

How we ran the test. We kept each test on logical CPU 0, though other work could still run on the same core. The runner checked each machine's cache layout before starting. We used addresses 4,096 bytes apart to reach the same set; Artemisia had 12 slots per set and the others had eight. The attacker and victim used different lines in separate parts of the same buffer. First we took 20,000 calibration readings (10,000 per bit) to choose a timing cutoff (threshold), which we kept unchanged for all later tests. We then ran ten batches of 10,000 trials for the main test, each with equal 0s and 1s, plus two control tests of 100,000 trials each – 320,000 saved trials per supported machine including calibration. We prepared memory before timing but did not run separate warm-up trials, and we shuffled the bits and the order of the two set measurements before each timed batch. For each set we timed each load separately and summed; our score was $D = (\text{time for set B}) - (\text{time for set A})$. We found the median score per bit during calibration and placed the cutoff halfway between them: above the cutoff meant bit 1, at or below meant bit 0. Scores use TSC ticks; we did not assume one tick equals one CPU cycle.

Main results. The median scores come from calibration; a score can be negative because it is a difference of two times. N is the number of test trials, N/A means no result. Each supported machine used 20,000 setup readings and 100,000 main-test trials. Thunderbird could not run this code.

Machine	Test trials	Train acc %	Test acc %	Med bit 0	Med bit 1	Cutoff
Sunbird	100,000	99.955	99.620	-60	48	-6
Charnwood	100,000	99.925	99.929	-26	34	4
Ookay	100,000	98.335	98.178	-24	32	4
Upgrade	100,000	99.440	99.511	-22	20	-1
Crux	100,000	94.465	94.066	-18	22	2
Artemisia	100,000	99.665	99.927	-128	120	-4
Skylark	100,000	51.795	51.842	0	0	0
Thunderbird	N/A	N/A	N/A	N/A	N/A	N/A

Table 10: Prime+Probe – setup readings and main-test accuracy. Median scores and cutoff in TSC ticks.

Machine	0 as 1 (FP)	1 as 0 (FN)	FP rate %	FN rate %	MI (bits/trial)
Sunbird	181	199	0.362	0.398	0.963983
Charnwood	16	55	0.032	0.110	0.991713
Ookay	15	1,807	0.030	3.614	0.884949
Upgrade	154	335	0.308	0.670	0.955915
Crux	16	5,918	0.032	11.836	0.725523
Artemisia	59	14	0.118	0.028	0.991556
Skylark	15,875	32,283	31.750	64.566	0.001098
Thunderbird	N/A	N/A	N/A	N/A	N/A

Table 11: Prime+Probe – wrong guesses and information learned. MI near 1 means the guesses reveal almost all of the bit.

Our code could not run on Thunderbird because it uses x86 timing instructions and Thunderbird is an ARM machine; its cache layout would also need a separate check. This is a limit of our code, not of whether Prime+Probe can work on ARM. Greenforest and Elevation were not part of this run.

Rate, time and memory. Correct bits/s is trials/s times accuracy; it counts correct guesses including lucky ones, so a high count near 50% accuracy is not useful leakage. Trial time includes priming, the victim, and both probes, excluding printing. Calibration time covers collecting the setup readings.

Machine	Trials/s	Correct bits/s	Med trial (ns)	Setup (ms)	Calib (ms)	Buffer (KiB)
Sunbird	1,208,155	1,203,564	792	0.921	17.231	64
Charnwood	1,889,842	1,888,500	508	0.426	10.843	64
Ookay	2,069,496	2,031,790	460	0.324	9.864	64
Upgrade	2,234,582	2,223,655	428	0.317	9.359	64
Crux	2,284,737	2,149,161	414	0.233	9.020	64
Artemisia	615,170	614,721	1,592	0.527	32.745	96
Skylark	991,147	513,830	962	0.409	20.031	64

Table 12: Prime+Probe – rate, time and memory. The attack buffer includes attacker and victim memory; each machine also used 800,000 bytes for recording and calibration arrays.

Timing distributions and trace. Sunbird shows clear separation between bits in the main test, with overlapping controls; Skylark's bit timings overlap. These plots show the middle 99% for readability; all samples were used for the reported numbers.

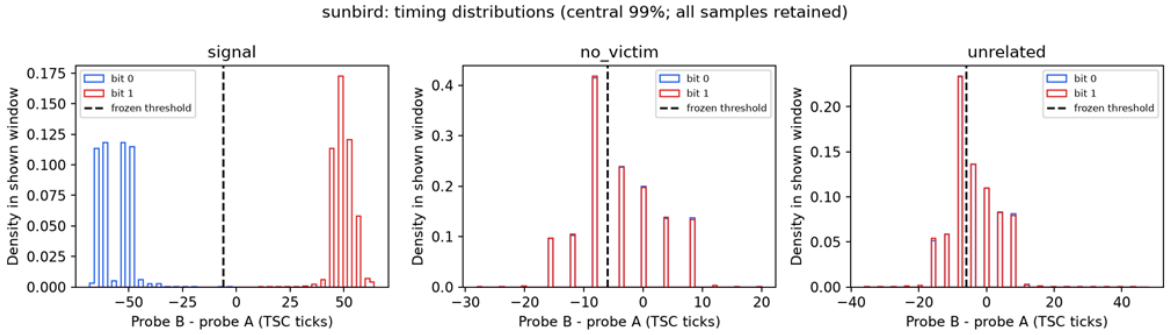


Figure 6: Sunbird: Prime+Probe timing distributions.

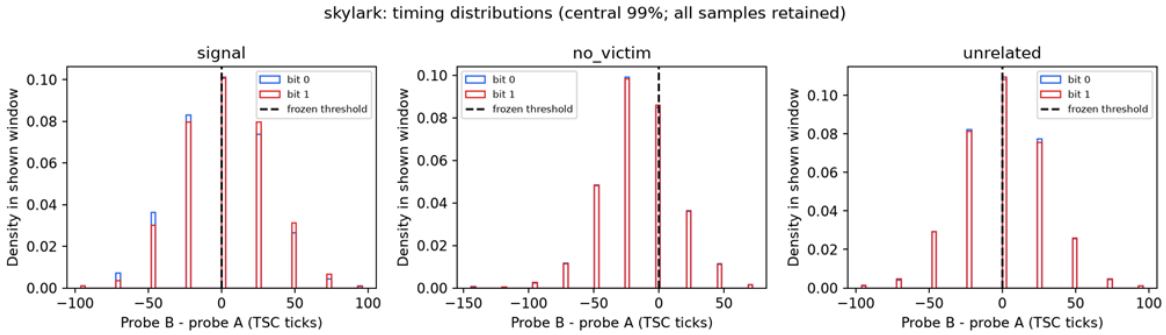


Figure 7: Skylark: Prime+Probe timing distributions (bit timings overlap).

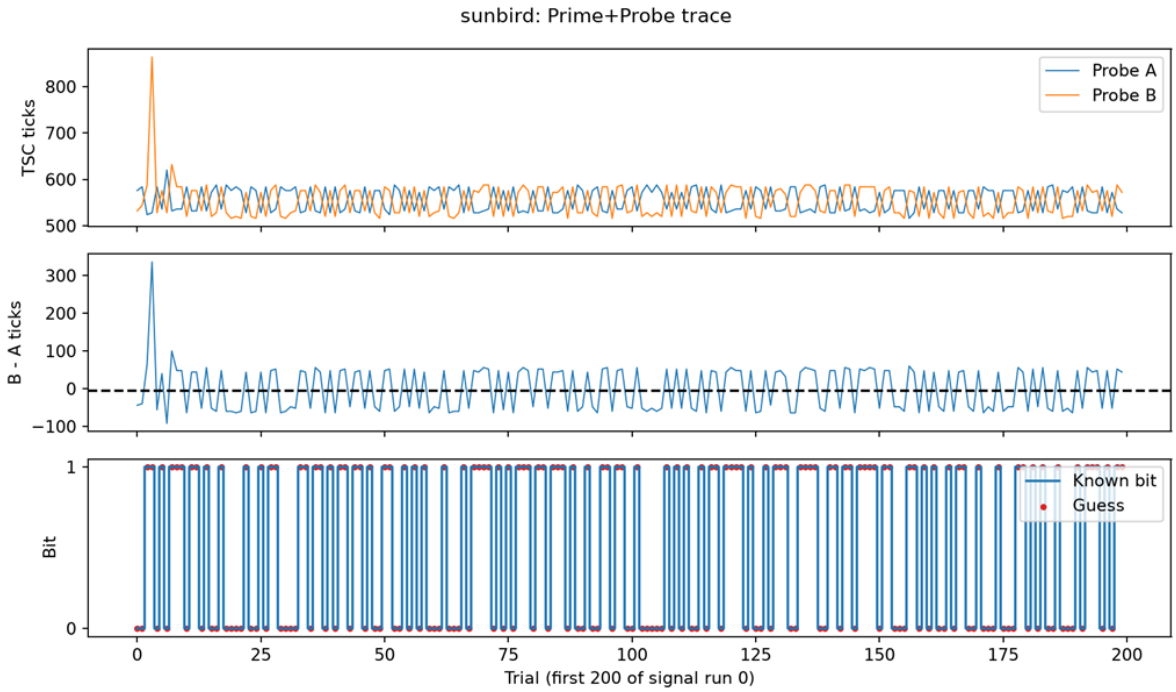


Figure 8: Sunbird: Prime+Probe trace, first 200 main-test trials from the first batch.

Defense comparison and cost (paper-based). This is a prediction, not a defense experiment. We compare Prime+Probe with Flush+Reload as required by Task 2.3; a defense added only to the LLC would not directly stop our L1 test. CEASER changes address-to-cache mappings with a secret key that changes over time, disrupting Prime+Probe if the mapping changes before the attacker learns useful conflicting addresses; it does not by itself stop Flush+Reload on a shared line. Its costs include encryption logic, shared control state, one tracking bit per line, and moving lines when the mapping changes (CEASER). ScatterCache uses secret mappings that depend on the security domain and cache way, making simple Prime+Probe address groups unreliable; separate cached copies of shared read-only data protect against cross-domain Flush+Reload on those copies, while shared writable data and flush permissions need extra handling. Its costs include mapping logic, per-line address information, and domain handling (ScatterCache). For scaling, assuming more sets with the same line size and ways, CEASER's line bits and full-cache remapping work grow with capacity (with 64-byte lines, one bit per line is about 0.195% of data capacity); adding ways instead increases candidate and mapping work. More storage and updates can cost area, time, and energy; neither defense has a proven fixed cost for every larger cache, and protection can weaken if attackers learn mappings faster (CEASER follow-up).

Shared replacement state and the cost of separating it. An LRU-style policy remembers which lines were used recently, and even a hit can change that record and affect later replacement, creating a timing channel. Separating programs' cache data is therefore not enough: replacement state and its updates also need separation (replacement-state study). Giving each program its own sets or ways reduces available space and adds control work; replacement-state storage grows with the sets and ways tracked, and resetting more state takes more work (clearing data adds refills and dirty writebacks). Clearing replacement bits alone leaves data occupancy visible. Our tests did not identify the exact replacement policy or measure these costs.

3.3 Flush+Reload

Flush+Reload uses the access time of a specific memory location to determine whether it was accessed by another process. In some cryptographic algorithms this can leak information about keys, as 1 or 0 bits access different stored values in the shared cache. To test viability, we built a benchmark using a probe function: it starts a timer with `rdtsc`, accesses a given address, stops the timer, then flushes the cache. If the location was already in memory the access is fast; if it was previously cached, the access is slow because the data must be loaded from DRAM. To set up, we ran a ∼100-access sequence to find the median hit/miss times, then set a threshold between the 90% high times of a hit and the 10% low times of a miss. We then ran a randomized benchmark for 20,000 accesses, where a tested location is randomly cached or evicted, and used the probe to predict whether it was accessed. In a real attack the attacker cannot directly control when to probe; instead they snoop the address repeatedly to detect a change in access times, build a model of how long to wait, and leak individual 1s or 0s.

Machine	N	Accuracy	Miss Latency	Hit Latency	False Pos.	False Neg.
Sunbird	20000	78.32%	296.54	166.24	4321	15
Charnwood	20000	76.85%	361.66	218.17	4561	70
Ookay	20000	76.71%	181.77	41.94	33	4626
Upgrade	20000	75.10%	189.29	53.72	801	4180
Crux	20000	81.06%	554.59	303.54	3788	0
Artemisia	20000	76.53%	286.33	158.86	4679	16
Greenforest	20000	99.98%	153.51	22.21	0	5
Elevation	20000	81.83%	209.15	51.85	133	3502

Table 13: Flush+Reload results. Latencies in TSC ticks.

3.4 Comparison and Contrast of the Three PoCs (2.2)

From our results, Prime+Probe had the best accuracy on every machine except Skylark, where it only reached 51% instead of the 99%+ elsewhere. Flush+Flush tended to have lower latencies than Flush+Reload but required a much longer training time, which makes it more vulnerable to detection from performance counters or OS/chip mechanisms. Flush+Reload achieved relatively high accuracy with a training cycle of only a few dozen memory accesses, making it stealthier.

Depending on the machine, Flush+Reload had either a high false-positive or a high false-negative rate, but not both, indicating a machine-dependent process either re-caching addresses that should have been flushed or evicting addresses that should have been accessed. Prime+Probe did not have this problem, achieving low false rates across the board because it relies on larger eviction sets that are not subject to noise from individual locations being cached or evicted. Flush+Flush showed results similar to Flush+Reload, again pointing to the high noise of single-line side channels.

Repeated runs across machines generally gave the same results as the tables, though there were occasional outliers, especially in Flush+Flush: sometimes the timing distributions for machines with too small a difference would be just different enough to give a good result, but this was not consistent, so we used the less accurate (more probable) result in the final tables. Raising the warm-up sample count to 10,000 helped on greenforest and elevation but made no difference elsewhere.

3.5 Predicted Defense Effectiveness and Cost as Caches Evolve (2.3)

CEASER is a cache defense that encrypts addresses before they are sent to the LLC. This "scatters" accesses, making it much harder for an attacker to determine access patterns. CEASER periodically

changes the cryptographic keys and remaps the cache gradually. It incurs a performance cost of only ∼1.1% and a storage overhead of less than 24 bytes. The author found an attacker takes about 22 seconds to learn the mapping of an 8 MB LLC, so remapping keys faster than this prevents the attacker from learning the mapping; breaking an 8 MB cache requires over 100 years of computation. CEASER actually increases in effectiveness as the cache grows, because the accesses needed to learn an eviction set are proportional to N^2 while the remap time is only N.

Against Prime+Probe, CEASER is very effective. Prime+Probe requires knowing which addresses conflict with a secret-dependent address in the LLC; if locations are randomized, the attacker must compute eviction sets fast enough to beat the remap time. CEASER can remap essentially arbitrarily fast (with a performance penalty), so the defense is arbitrarily robust against Prime+Probe. Against Flush+Reload, CEASER does nothing: Flush+Reload measures the access time of a known address, and randomizing that address's physical location does not change the attack.

ScatterCache randomizes the index location of a cache line based on the Security Domain ID of each process, so for any access the set is a random selection of lines from all available lines. This makes eviction sets extremely difficult to determine, forcing Prime+Probe to need over 33.5 million accesses instead of 103 for a 16 KB cache. Prime+Probe is still possible against ScatterCache – it takes far more accesses, but eviction sets can still be constructed – whereas Flush+Reload is essentially completely blocked, because the index depends on the process ID and the attacker can never access the same physical location as the victim.

CEASER should remain low overhead as cache sizes rise: it relies on a per-cache hardware encrypting unit that does not depend on cache size, and its performance cost depends only on the encryption time. The only area where performance could decrease is as remapping happens more often, but the remap cost is very small. ScatterCache grows in complexity with associativity: if caches become more highly associative, the hardware cost to compute addresses rises because of a per-way lookup on each access. In the future ScatterCache will be slightly less effective than now (higher performance cost) but will still be a strong defensive layer.

4 Evidence of Data-Memory-Dependent Prefetching (DMP)

4.1 Detection Test and Setup

To test for the DMP, a 64-wide array was filled with pointers to cache lines spread over 1024 pages, and all the targets were flushed. The first 32 entries were then read as integers to prevent dereferencing. Then a target in the second half of the array was timed; if that comes back as a hit, a DMP is likely, due to the DMP prefetching the target based on the pointer value stored in memory.

Across the eight lab machines, the test was run pinned to a core (a P-core on hybrid parts). The tests were repeated twice and the results uploaded. The full detection code is in Appendix B; build and run with:

```
gcc -O2 -o dmp_find dmp_find.c find_threshold.c
taskset -c <core> ./dmp_find   # core = a P core on hybrid parts
cat dmp_results.txt
```

4.2 Evidence and Measurements

The only machine showing DMP existence was Greenforest, a Raptor Lake part. Every other machine did not behave in a way that would indicate DMP presence.

Machine	CPU	Microarch.	Pointer vs. Garbage	Peak Pointer-Distance Hit Rate %	DMP?
Greenforest	i7-14700	Raptor Lake	2.2–3.7% vs. .01%	26–50% @ d=1	Yes
Artemisia	Xeon Gold 5420+ ×2	Sapphire Rapids	.01–.35% vs. 0%	.9% @ d=8	No
Elevation	i5-1235U	Alder Lake	.01% vs. 0%	.16% @ d=7	No
Crux	i7-9700	Coffee Lake	.01% vs. 0%	.17% @ d=22	No
Upgrade	i7-8700	Coffee Lake	.01% vs. 0%	.16% @ d=7	No
Ookay	i7-7700	Kaby Lake	.01% vs. 0%	.17% @ d=15	No
Charnwood	i7-6700	Skylake	0–.01% vs. 0–.01%	.28% @ d=30	No
Sunbird	Xeon E5-2680v3 ×2	Haswell	.01–.03% vs. 0%	.17% @ d=22	No

Table 14: Summary of results for the eight machines.

The "Pointer vs. Garbage" and "Peak Pointer-Distance Hit Rate" columns show the key insights. The first compares the overall hit rate, across all 32 distances, of the test (pointer) against the control (garbage). In the garbage control, only the probed entry is replaced with a non-pointer value; every other entry holds the same pointers as the test. The range is shown across runs. The second column breaks this down by distance (which of the 32 unread entries, 32 to 63, was probed), showing the highest pointer rate per machine and the garbage rate at that distance.

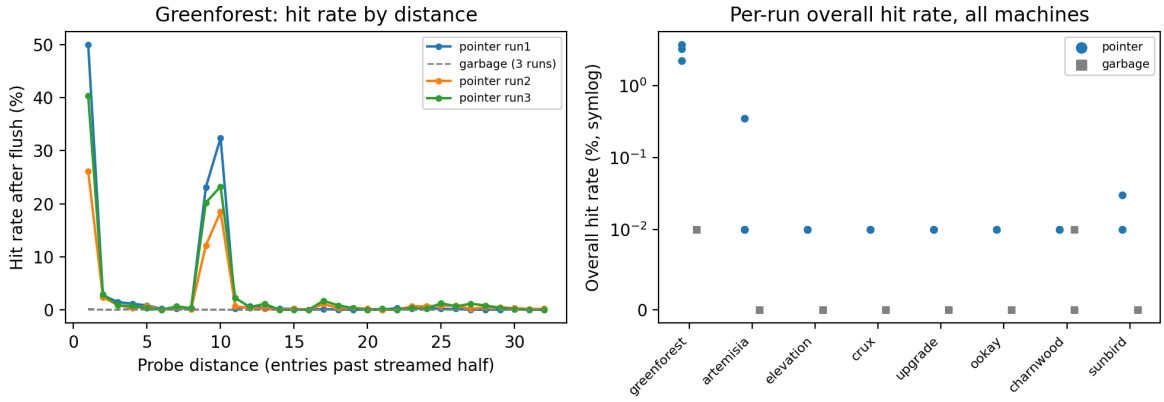


Figure 9: Left: Greenforest hit rate after flush by probe distance, pointer (solid) vs. garbage (dashed). Right: overall hit rate per run for every machine.

Machine	Run	Thr (cyc)	SW-pf %	Ptr %	Garb %	Best d	Ptr @d %	Garb @d %
Greenforest	1	36	100.00	3.68	0.01	1	50.00	0.15
Greenforest	2	46	100.00	2.21	0.01	1	26.12	0.00
Greenforest	3	35	100.00	3.23	0.01	1	40.30	0.15
Artemisia	1	104	99.93	0.01	0.00	7	0.16	0.00
Artemisia	2	103	99.97	0.01	0.00	7	0.16	0.00
Artemisia	3	103	100.00	0.35	0.00	8	0.91	0.00
Elevation	1	65	99.99	0.01	0.00	7	0.16	0.00
Elevation	2	64	98.66	0.01	0.00	7	0.16	0.00
Elevation	3	29	98.66	0.01	0.00	7	0.16	0.00
Crux	1	30	100.00	0.01	0.00	22	0.17	0.00
Crux	2	29	100.00	0.01	0.00	22	0.17	0.00
Crux	3	30	100.00	0.01	0.00	7	0.16	0.00
Upgrade	1	63	100.00	0.01	0.00	7	0.16	0.00
Upgrade	2	59	100.00	0.01	0.00	7	0.16	0.00
Upgrade	3	61	100.00	0.01	0.00	7	0.16	0.00
Ookay	1	68	99.90	0.01	0.00	7	0.16	0.00
Ookay	2	68	100.00	0.01	0.00	7	0.16	0.00
Ookay	3	69	99.98	0.01	0.00	15	0.17	0.00
Charnwood	1	70	100.00	0.01	0.01	13	0.16	0.00
Charnwood	2	73	100.00	0.01	0.00	7	0.16	0.00
Charnwood	3	119	99.99	0.01	0.00	7	0.16	0.00
Sunbird	1	70	100.00	0.01	0.00	7	0.16	0.00
Sunbird	2	69	100.00	0.01	0.00	12	0.16	0.00
Sunbird	3	72	100.00	0.03	0.00	16	0.17	0.00

Table 15: Raw results for every run. SW-pf = software-prefetch T0 control hit rate; Ptr/Garb = overall pointer and garbage hit rates; Best d = distance with the largest pointer-minus-garbage gap, with the pointer and garbage rates at that distance.

In all three Greenforest runs the pointer condition produced cache hits on lines that were flushed and never read (2.2% to 3.7% overall, 26% to 50% at the best distance). The garbage control, with identical addresses and flushes and only the probed array value changed, stayed at 0.01% overall and at most 0.16% at any distance. The 20,000 trials per condition make a gap of this size very unlikely to be noise. The software-prefetch controls were at 99.99% to 100%, so the probe and threshold can detect a prefetched line, and misses on random lines were separated from hits via the calibrated threshold. The hits concentrate at distances 1, 9, and 10 (Figure 9, left), about 50%, 23%, and 32% in run 1, and the same shape repeats in runs 2 and 3. A flush failure, threshold error, or generic stride/stream prefetcher would not depend on the value stored in one array entry and would show up in the garbage control too; this dependence on memory contents is what a data memory-dependent prefetcher does.

On the other seven machines the pointer and garbage conditions are indistinguishable. With about 625 trials per distance, a single hit is 0.16%, so the "best distance" values of 0.16% to 0.17% in Table 15 are one stray hit each, with no consistent distance across runs, and the overall pointer rate is 0.01% (2 hits in 20,000), equal to the garbage floor. The few larger values are isolated: Artemisia run 3 (0.35% overall, 0.91% at d=8) and Sunbird run 3 (0.03%) did not reproduce on the same machine and are about 1 to 6 hits. The software-prefetch controls were 98.4% to 100% on every machine, so the probe could have seen a prefetched line had one existed. These runs give no evidence of a DMP under these conditions; they do not prove its absence.

4.3 Conclusion, Limitations, and Relevance to Homework 1

On the evidence above, only Greenforest (Raptor Lake) is classified as **DMP observed**; the other seven are **not observed under the tested conditions**. A missing signal does not prove absence. This test probes only one access pattern: 64-entry arrays of full 64-bit pointers read sequentially as integers, targets on random lines of a 4 MB pool, a fixed delay of 1000 loop iterations, and a single read-then-wait sequence. A DMP that needs a different training pattern, more repetitions, a higher confidence threshold, a different target region, or a longer delay would be missed. Hit/miss classification also depends on a per-run threshold, and the noisier machines have less margin. The test shows that on Greenforest the cache state of a line depends on the contents of the array, but not which prefetcher is responsible, and it does not rule out an unrelated content-dependent mechanism.

Homework 1 results could be affected because DMP presence means a flushed line can become cached again without the program touching it whenever nearby memory contains pointer-like values. Any Homework 1 cache-timing measurement on Greenforest that flushed lines and later timed them, with neighboring pointer-like data, could show incorrect hits and affect measured hit/miss rates and latencies. The seven machines with no observed DMP did not show this effect in the tested setup, so their Homework 1 numbers are less likely to be affected.

5 Discussion and Conclusion

5.1 Relevance of the Findings to Different Stakeholders

Reverse engineering a cache is not a single-audience exercise, and the results in Homework 1 and 2 carry a different value depending on who is reading them. Interested parties include attackers, who want a precise picture of the hierarchy, its latencies, and its sharing behavior to know which side channels to target. Equally important are defenders, hardware vendors, and even students, and the same measurements that help an attacker tell them something useful in return. These homeworks can also present attackers and defenders an opportunity for changing their scope of optimization and business strategy.

For hardware vendors, the findings are mostly strategic. A comparison across a decade of parts shows them where there is still room to grow and where a wall has been reached. The configuration space here is enormous, so a setting known to work well is worth protecting and is valuable to companies that no longer have to spend R&D time to find it. The protection is partly about intellectual property and partly about security, and it is one reason a manufacturer would want reverse engineering to be harder than on a competitor's part. We saw a concrete version of this: AMD's L3 was difficult to observe in full because its chiplet architecture only exposes the slice of L3 on your own die, with latency jumping sharply past it, and AMD timers were harder to serialize and convert back to nanoseconds than the other x86 parts. Lower observability lends itself as a defensive property.

The Flush+Flush study gave the clearest example of a finding directly actionable for both attackers and defenders. We found that if the machine was not warmed up enough it stayed in a low-power state, and the default run of about 1,000 warm-up samples was far too short. The attack timing read roughly five to six times too high during that cold window, and the threshold learned there did not transfer, so accuracy collapsed. Once we added 10,000 warm-up samples, elevation's single-process run went from 50.0% to 100.0% test accuracy, greenforest's multithreaded run went from 45.6% to 80.7%, and greenforest's single-process run improved from 50.0% to 63.2%. The cross-thread attack at best was around 70 to 77% on the best machines, and on several others the gap was so small that a 50/50 split guessed one class and landed at the 50% random baseline,

essentially saying Flush+Flush was useless there. A defender who understands this can add timing noise or shift the threshold to make the attack harder without closing it entirely, and a software defense like CEASER, which randomizes cache mapping, becomes a reasonable mitigation to layer on top.

The organizational trends feed design decisions too, and they connect back to H1 and H2. Hill's prediction that a direct-mapped L1 would be enough assumed the miss penalty would never grow large enough to need associativity and that extra comparators would slow the hit path; both assumptions were overtaken, the first because programs grew and L1 limits were reached, and the second because multi-port RAM and better front-end circuits removed the latency cost he worried about. As L1 saturated, attention moved to a larger L2, exactly where an aggressive stride prefetcher became worthwhile, since a bigger L2 can absorb the pollution that would thrash a small one. Which prefetcher to insert still depends on the workload – a pointer-chasing program wanting a data-dependent prefetcher while random accesses want a conventional one – the same application-dependence McKee described for the memory wall in H2. The measurements do not just expose the hardware; they hint at where the next prefetcher should go and for whom.

Furthermore, students can use this information if they want to get into reverse-engineering microarchitecture. Homework 1 and Homework 2 provide a variety of experiments that form a basis for cache experimentation and PoCs, and if done properly also include methodology and implementation details – accurate enough that a student (or any interested party) could take the code, follow setup instructions, and recreate the attack or experiment. It could also serve as a launch pad for future research. Each baseline had to establish hypotheses and make "laws" about the future of cache implementation; a future researcher, several generations later, could repeat this homework and see which trends held, which did not, and why.

5.2 Discussion Notes: Proposals, Objections, and Evidence

Proposal: Normalize all cross-ISA latency to nanoseconds using measured per-machine tick rates before comparing machines (Challenge 1).
Objection: Raw ticks are a standardized collection format and are fine within a single machine; conversion adds a calibration dependency.
Evidence: Without conversion, the ARM part is a spurious outlier; after ns normalization it aligns with the x86 parts (Section 2.1).

Proposal: Treat the Flush+Flush threshold as warm-up dependent and require a long warm-up (or per-phase re-calibration).
Objection: A longer warm-up raises setup cost, may be unavailable to a real attacker, and does not fully close the channel.
Evidence: Adding +10k warm-up moved elevation from 50.0% to 100.0% (single) and greenforest from 45.6% to 80.7% (multi); the untreated cold window read 5–6× high (Tables 8, 9).

Proposal: Classify Greenforest (Raptor Lake) as exhibiting DMP and the older generations as not, under the tested conditions.
Objection: Three trials and P-/E-core pinning limit confidence; absence of a signal is not proof of absence of a DMP.
Evidence: Greenforest's pointer hit rate (2.2–3.7%, peak 26–50% at $d = 1$) separates cleanly from its garbage control (.01%); all other machines match their controls (Tables 14, 15).

Proposal: Expect CEASER to stay low-overhead as caches grow while ScatterCache's cost rises with associativity (Section 3.5).

Objection: Neither defense has a proven fixed cost for every larger cache, and protection can weaken if attackers learn mappings faster.
Evidence: CEASER's eviction-set learning scales as N^2 vs. remap N; ScatterCache forces >33.5M accesses vs. 103 for a 16 KB cache, but needs a per-way lookup per access.

5.3 Contributions and Meetings

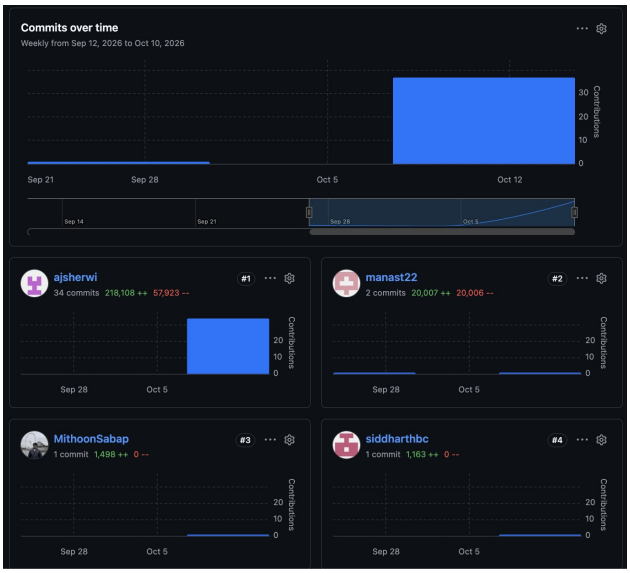


Figure 10: Contribution Graph

Manas: Finished 2 of 5 challenges, graded 4 baselines, completed the AI-usage analysis (Section 1.2), worked on H1 and H2, and implemented the Flush+Flush PoC. Also synthesized PoCs for Section 2.2 and wrote the final stakeholder-relevance page. Worked on the report, slides, and video presentation.

Abbott: Described 1 of 5 challenges, graded 5 baselines, implemented the Flush+Reload PoC. Also synthesized PoCs for Section 2.2 and completed Section 2.3 (comparing defenses with 2 of 3 PoCs). Worked on the report, slides, and video presentation.

Mithoon: Led the DMP investigation (implementation, results, and conclusions for all of Section 3), including code and methodology and the per-machine DMP verdicts with supporting evidence. Also did 1 of 5 challenges and graded 4 baselines (Section 1.2). Worked on the report, slides, and video presentation.

Siddarth: Finished 1 of 5 challenges and graded 4 baselines for Section 2.2. Worked on H3 (a trend that may not last) and the SHiP++ discussion (Section 1.3), implemented Prime+Probe, and contributed to the Section 2.2 synthesis. Worked on the report, slides, and video presentation.

Disclaimer: the contribution record indicates who was in the lead role for each item; it does not mean other members had no role. Each task had a lead member with support from the rest of the team.

Team meetings were held in Koch Hall, Lab 1014, about 2–3 times per week while the project was live, for roughly 2–3 hours each, working on individual items while helping each other to strengths.

Allowed AI usage: The only AI used by any member was for formatting the LaTeX report and slides – cleaning up the report and building a clean document using only conclusions, data,

and writing the team had done previously (tables, graphs, paragraph spacing, section titling, and layout). No new content was generated by AI: all work was done originally by team members, and then cleaned up in formatting only by AI.

References

1. M. D. Hill, "A Case for Direct-Mapped Caches," *IEEE Computer*, 1988. (H1)
2. S. A. McKee, "Reflections on the Memory Wall," *Computing Frontiers*, 2004. (H2)
3. N. P. Jouppi, "Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers," *ISCA*, 1990. (H3)
4. V. Young, C.-C. Chou, A. Jaleel, and M. K. Qureshi, "SHiP++: Enhancing Signature-Based Hit Predictor for Improved Cache Performance," *2nd Cache Replacement Championship*, 2017. [38] – paper, code.
5. D. Gruss *et al.*, "Flush+Flush: A Fast and Stealthy Cache Attack," DIMVA, 2016. [35]
6. Y. Yarom and K. Falkner, "Flush+Reload: A High Resolution, Low Noise, L3 Cache Side-Channel Attack," USENIX Security, 2014. [25]
7. D. A. Osvik, A. Shamir, and E. Tromer, "Cache Attacks and Countermeasures: the Case of AES" (Prime+Probe), CT-RSA, 2006. [26]
8. M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," MICRO, 2018. [27] – link; CEASER-S follow-up.
9. M. Werner *et al.*, "ScatterCache: Thwarting Cache Attacks via Cache Set Randomization," USENIX Security, 2019. [28] – link.
10. Replacement-state leakage study, IEEE TC, 2021. [29]

A Source Code Listings

A.1 Flush+Flush – single thread (`ff_simple.c`)

```
#include <stdio.h>
#include <stdint.h>
#include <x86intrin.h>

volatile char target = 85; //target byte that we are trying to observe with ff

uint64_t time_flush(volatile char *addr) { //basic function for running ff
    _mm_mfence(); _mm_lfence(); //set up memory and time fence for any previous instructions
    uint64_t start = __rdtsc(); //use x86 rdtsc for starting timing measurement
    _mm_lfence(); //one more lfence to make sure timing function finishes
    _mm_clflush((const void *)addr); //actual clflush for attack
    _mm_mfence(); _mm_lfence(); //time/memory fence to ensure singularity
    uint64_t end = __rdtsc(); //end timer
    return end - start; //calculate delta of flush instruction
}

uint64_t trial(int touch) { //trial function for finding baseline times
    if (touch) { volatile char c = target; (void)c; } //if touched previously, set this as target pull into
     cache
    return time_flush(&target); //flush attack target for present and not present- get time
```

```
}


void collect(uint64_t *times, int *touched, int n, int warmup) {
    for (int i = 0; i < warmup; i++) trial(i & 1);   // warm up, discard
   for (int i = 0; i < n; i++) {
        touched[i] = i & 1; //alternate even and odd (so 50/50 split between cached and not)
        times[i] = trial(touched[i]); //initialize and time each trial set by N, i
        //basically this function calls flush and flush N-1 times to learn overall sets

   }
}

void sort_u64(uint64_t *a, int n) { //sorting collected times to find median (insertion sort)
    for (int i = 1; i < n; i++) {
        uint64_t key = a[i]; //sorting head
        int j = i - 1; //secondary index
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; } //slide array[i] left until it is smallest
        a[j + 1] = key; //increment key
    }
}

uint64_t median_sorted(const uint64_t *sorted, int n) { //find median from sorted array, decided on median
     after sensitivity of mean
    if (n % 2) return sorted[n / 2]; //if array elements are odd, return exact middle (stepped)
    return (sorted[n / 2 - 1] + sorted[n / 2]) / 2; //else return middle of two middle values
}

void learn(const uint64_t *times, const int *touched, int n,
           uint64_t *threshold, int *cached_is_slower, FILE *file) { //function to learn from calibration set
      for future test
    uint64_t cached[n], uncached[n]; //split training data into cached and uncaches
    int n_c = 0, n_u = 0; //number of hits index
    for (int i = 0; i < n; i++) {
        if (touched[i]) cached[n_c++]   = times[i]; //actually placing timing data into arrays
        else            uncached[n_u++] = times[i];
    }
    sort_u64(cached, n_c);
    sort_u64(uncached, n_u); //sort

    uint64_t med_c = median_sorted(cached, n_c); //find median
    uint64_t med_u = median_sorted(uncached, n_u);//find mediam
    *threshold = (med_c + med_u) / 2; //find threshold by citing middle ground between cached and uncached
    *cached_is_slower = med_c > med_u; //is cached time to flush slower than uncached (NO(0) is usually the
     right sign)
    printf("TRAINING METRICS: median cached=%llu  median uncached=%llu  " //print all collected statistics
     that we are measuring against
           "threshold=%llu  cached_is_slower=%d\n",
           (unsigned long long)med_c, (unsigned long long)med_u,
           (unsigned long long)*threshold, *cached_is_slower);
    fprintf(file, "TRAINING METRICS: median cached=%llu  median uncached=%llu  " //print all collected
     statistics that we are measuring against
           "threshold=%llu  cached_is_slower=%d\n",
           (unsigned long long)med_c, (unsigned long long)med_u,
           (unsigned long long)*threshold, *cached_is_slower);



}

int score(const uint64_t *times, const int *touched, int n,
          uint64_t threshold, int cached_is_slower, FILE *file) { //actual prediction function, takes in
     metrics from training, against real data
    int correct = 0; //how many guesses correct
    for (int i = 0; i < n; i++) {
        int guess = cached_is_slower ? (times[i] > threshold) //ternary to decide if cache is slower, guess
     accurately based on timing threshold
                                     : (times[i] < threshold);
```

```
        correct += (guess == touched[i]); //use this "known state" array to grade guess
        fprintf(file, "%llu,%d, %d\n", times[i], guess, touched[i]);
    }

    return correct; //return number of correct guesses

}

int main(void) {

    FILE *file = fopen("output_ff.csv", "w");
    FILE *file_txt = fopen("output_ff.txt", "w");

    if (file == NULL) {
        printf("Failed to open file output.csv\n");
    } else {
        printf("File Opened\n");
    }

    fprintf(file, "latency,prediction,actual\n");


    const int N = 10000, WARMUP = 1000; //set training samples and test samples
    uint64_t train_start = __rdtsc();
    uint64_t train_times[N]; int train_touched[N]; //est data sets for training times and predetermined
     cached/uncached
    collect(train_times, train_touched, N, WARMUP); //set these data sets for usage

    uint64_t threshold; int cached_is_slower; //declare threshold vars
    learn(train_times, train_touched, N, &threshold, &cached_is_slower, file_txt); //set threshold vars based
      on learning function
    int train_correct = score(train_times, train_touched, N, threshold, cached_is_slower, file);
    uint64_t train_end = __rdtsc();
    uint64_t train_tot = train_end - train_start;
    //baseline check to make sure scoring is accruate on data WE KNOW IS GOOD

        uint64_t test_start = __rdtsc();

    uint64_t test_times[N]; int test_touched[N]; //declare new data sets for actual test
    collect(test_times, test_touched, N, WARMUP); //set these data sets for usage

    int test_correct = score(test_times, test_touched, N,threshold, cached_is_slower, file);
    //NOW we can score WITHOUT learning to see if we can use previous calibration to do F + F
         uint64_t test_end = __rdtsc();
    uint64_t test_tot = test_end - test_start;
    printf("\ntrain accuracy: %d/%d = %.1f%%\n", train_correct, N, 100.0 * train_correct / N); //print out
     training results
    printf("test accuracy:  %d/%d = %.1f%%\n", test_correct, N, 100.0 * test_correct / N); //print out real
     test results
    printf("train time: %llu, test time: %llu\n", (unsigned long long)train_tot, (unsigned long long)test_tot
     );

    fprintf(file_txt,"\ntrain accuracy: %d/%d = %.1f%%\n", train_correct, N, 100.0 * train_correct / N); //
     print out training results
    fprintf(file_txt,"test accuracy:  %d/%d = %.1f%%\n", test_correct, N, 100.0 * test_correct / N); //print
     out real test results
    fprintf(file_txt,"train time: %llu, test time: %llu\n", (unsigned long long)train_tot, (unsigned long
     long)test_tot);

    return 0;
}
```

A.2 Flush+Flush – multi thread (`ff_mthread.c`)

```
// ff_mthread.c Flush+Flush with separate threads
//
```

```
// Build: gcc -O0 -march=native -pthread ff_mthread.c -o ff_thread
// Run:   ./ff_thread


#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <x86intrin.h>

volatile char target = 85; //target byte that we are trying to observe with ff

#define N      10000  //samples and warmup
#define WARMUP  1000


static pthread_barrier_t round_start; //barriers to ensure ordering of threads
static pthread_barrier_t victim_done;


typedef struct {
    uint64_t *times;   // attacker writes here
    int      *touched; // victim writes here
    int       n; //samples
    int       warmup; //calibration/jitter
    uint32_t  seed;    // victim uses this; attacker ignores it
} thread_args;

uint64_t time_flush(volatile char *addr) { //basic function for running ff
    _mm_mfence(); _mm_lfence(); //set up memory and time fence for any previous instructions
    uint64_t start = __rdtsc(); //use x86 rdtsc for starting timing measurement
    _mm_lfence(); //one more lfence to make sure timing function finishes
    _mm_clflush((const void *)addr); //actual clflush for attack
    _mm_mfence(); _mm_lfence(); //time/memory fence to ensure singularity
    uint64_t end = __rdtsc(); //end timer
    return end - start; //calculate delta of flush instruction
}

void sort_u64(uint64_t *a, int n) { //sorting collected times to find median (insertion sort)
    for (int i = 1; i < n; i++) {
        uint64_t key = a[i]; //sorting head
        int j = i - 1; //secondary index
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; } //slide array[i] left until it is smallest
        a[j + 1] = key; //increment key
    }
}

uint64_t median_sorted(const uint64_t *sorted, int n) { //find median from sorted array, decided on median
     after sensitivity of mean
    if (n % 2) return sorted[n / 2]; //if array elements are odd, return exact middle (stepped)
    return (sorted[n / 2 - 1] + sorted[n / 2]) / 2; //else return middle of two middle values
}

void learn(const uint64_t *times, const int *touched, int n,
           uint64_t *threshold, int *cached_is_slower, FILE *file) { //function to learn from calibration set
      for future test
    uint64_t cached[n], uncached[n]; //split training data into cached and uncached
    int n_c = 0, n_u = 0; //number of hits index
    for (int i = 0; i < n; i++) {
        if (touched[i]) cached[n_c++]   = times[i]; //actually placing timing data into arrays
        else            uncached[n_u++] = times[i];
    }
    sort_u64(cached, n_c);
    sort_u64(uncached, n_u); //sort

    uint64_t med_c = median_sorted(cached, n_c); //find median
    uint64_t med_u = median_sorted(uncached, n_u); //find median
```

```
    *threshold = (med_c + med_u) / 2; //find threshold by citing middle ground between cached and uncached
    *cached_is_slower = med_c > med_u; //is cached time to flush slower than uncached (NO(0) is usually the
     right sign)
    printf("TRAINING METRICS: median cached=%llu  median uncached=%llu  " //print all collected statistics
     that we are measuring against
           "threshold=%llu  cached_is_slower=%d\n",
           (unsigned long long)med_c, (unsigned long long)med_u,
           (unsigned long long)*threshold, *cached_is_slower);
    fprintf(file, "TRAINING METRICS: median cached=%llu  median uncached=%llu  "
           "threshold=%llu  cached_is_slower=%d\n",
           (unsigned long long)med_c, (unsigned long long)med_u,
           (unsigned long long)*threshold, *cached_is_slower);
}

int score(const uint64_t *times, const int *touched, int n,
          uint64_t threshold, int cached_is_slower, FILE *file) { //actual prediction function, takes in
     metrics from training, against real data
    int correct = 0; //how many guesses correct
    for (int i = 0; i < n; i++) {
        int guess = cached_is_slower ? (times[i] > threshold) //ternary to decide if cache is slower, guess
     accurately based on timing threshold
                                     : (times[i] < threshold);
        correct += (guess == touched[i]); //use this "known state" array to grade guess
        fprintf(file, "%lu,%d,%d\n", times[i], guess, touched[i]); //print to csv
    }
    return correct; //return number of correct guesses
}


void *victim(void *arg) {
    thread_args *a = (thread_args *)arg; //set arguments for victim thread based on set params (have to pass
     in as void and recast based on documentation for ptthread)
    uint32_t seed = a->seed; //set seed

    for (int round = 0; round < a->warmup + a->n; round++) {
        pthread_barrier_wait(&round_start); // sync for victim is about to act event

        seed ^= seed << 13; seed ^= seed >> 17; seed ^= seed << 5;
        int touch = seed & 1; // flip a coin -- attacker doesn't know this
        if (touch) { volatile char c = target; (void)c; } //bring in target into shared space if touch ==
     true

        pthread_barrier_wait(&victim_done); // signal that victim has acted and now attacker can time

        if (round >= a->warmup) // start recording after warmup (may avoid early issues)
            a->touched[round - a->warmup] = touch; //fill out touched array to check guesses later
    }
    return NULL;
}


void *attacker(void *arg) {
    thread_args *a = (thread_args *)arg; //again pass args for attacker based on params

    for (int round = 0; round < a->warmup + a->n; round++) {
        pthread_barrier_wait(&round_start); // sync for victim is about to act event
        pthread_barrier_wait(&victim_done); // wait for victim, has now acted and can time flush

        uint64_t t = time_flush(&target); // probe shared memory, and time the clflush

        if (round >= a->warmup) // start recording after warmup (may avoid early issues)
            a->times[round - a->warmup] = t; //fill out time for each access
    }
    return NULL;
}

// Run one phase (train or test): init barriers, start both threads,
// wait for both to finish, then destroy barriers.
```

```
static void run_phase(uint64_t *times, int *touched, uint32_t victim_seed) {
    pthread_barrier_init(&round_start, NULL, 2); // 2 threads must arrive for barrier to release
    pthread_barrier_init(&victim_done, NULL, 2); // same for victim doen as well

    pthread_t t_victim, t_attacker; //establish processes for victim and attacker
    thread_args args = { times, touched, N, WARMUP, victim_seed }; //set args for struct defined earlier
    pthread_create(&t_victim,   NULL, victim,   &args); //create addresses for attacker and victim threads
     and call processes
    pthread_create(&t_attacker, NULL, attacker, &args);
    pthread_join(t_victim,   NULL); //join these, so threads must wait on each other to move on
    pthread_join(t_attacker, NULL);

    pthread_barrier_destroy(&round_start); //frees barriers at reference address, so process can move on to
     next iter
    pthread_barrier_destroy(&victim_done);
}

int main(void) {

    FILE *file     = fopen("output_ff_multi.csv", "w"); //set output files (data raw csv and measured txt)
    FILE *file_txt = fopen("output_ff_multi.txt", "w");

    if (file == NULL) { //error collection for issue faced
        printf("Failed to open file output_ff.csv\n");
    } else {
        printf("File Opened\n");
    }

    fprintf(file, "latency,prediction,actual\n"); // set csv headers

    uint64_t train_times[N]; int train_touched[N]; //data sets for training times and predetermined cached/
     uncached
    uint64_t test_times[N];  int test_touched[N];  //declare new data sets for actual test

    uint64_t train_start = __rdtsc(); //simple timing prim for testing train vs run (train should take longer
      bc of calibration)
    run_phase(train_times, train_touched, 98765); // set run with seed

    uint64_t threshold; int cached_is_slower; //declare threshold vars
    learn(train_times, train_touched, N, &threshold, &cached_is_slower, file_txt); //set threshold vars based
      on learning function
    int train_correct = score(train_times, train_touched, N,
                              threshold, cached_is_slower, file); //baseline check on data WE KNOW IS GOOD
    uint64_t train_end = __rdtsc(); //timing prim end
    uint64_t train_tot = train_end - train_start; //timing delta


    uint64_t test_start = __rdtsc(); //timing prim for test phase
    run_phase(test_times, test_touched, 11111); //start run

    int test_correct = score(test_times, test_touched, N,
                             threshold, cached_is_slower, file); //score WITHOUT re-learning to test if
     calibration holds
    uint64_t test_end = __rdtsc();
    uint64_t test_tot = test_end - test_start; //timing primitive delta

    // Results
    printf("\ntrain accuracy: %d/%d = %.1f%%\n", train_correct, N, 100.0 * train_correct / N);
    printf("test accuracy:  %d/%d = %.1f%%\n",  test_correct,  N, 100.0 * test_correct  / N);
    printf("train time: %llu cycles, test time: %llu cycles\n",
           (unsigned long long)train_tot, (unsigned long long)test_tot); //print results to terminal

    fprintf(file_txt, "\ntrain accuracy: %d/%d = %.1f%%\n", train_correct, N, 100.0 * train_correct / N);
    fprintf(file_txt, "test accuracy:  %d/%d = %.1f%%\n",  test_correct,  N, 100.0 * test_correct  / N);
    fprintf(file_txt, "train time: %llu cycles, test time: %llu cycles\n",
           (unsigned long long)train_tot, (unsigned long long)test_tot); //print results to file

    fclose(file);
```

```
    fclose(file_txt);//close files
    return 0;
}
```

A.3 Prime+Probe (pp_measure.c)

```
/* Prime+Probe measurements. Build and run instructions are in README.txt.
 * stdout: samples; stderr: setup, calibration and batch timings. */
#define _GNU_SOURCE
#include <cpuid.h>
#include <inttypes.h>
#include <sched.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <time.h>
#include <unistd.h>
#include <x86intrin.h>

#define MAX_WAYS 64

typedef struct {
    uint64_t a, b, ns;
    int bit, order;
} Sample;

static int ways, line;
static size_t stride;
static uint8_t *memory;

static uint64_t now_ns(void) {
    struct timespec t;
    if (clock_gettime(CLOCK_MONOTONIC_RAW, &t)) {
        perror("clock_gettime");
        exit(1);
    }
    return (uint64_t)t.tv_sec * 1000000000ULL + (uint64_t)t.tv_nsec;
}

// Fence the load between timestamps.
static uint64_t time_load(volatile uint8_t *address) {
    unsigned int aux;
    _mm_lfence();
    uint64_t start = __rdtsc();
    _mm_lfence();
    (void)*address;
    _mm_lfence();
    uint64_t end = __rdtscp(&aux);
    _mm_lfence();
    return end - start;
}

// Stride = sets * line size, keeping the set index fixed.
static void prime(volatile uint8_t *base) {
    for (int i = 0; i < ways; ++i)
        (void)base[i * stride];
}

static uint64_t probe(volatile uint8_t *base) {
    uint64_t ticks = 0;
    for (int i = 0; i < ways; ++i)
        ticks += time_load(base + i * stride);
    return ticks;
}
```

```
// 0: signal, 1: no reads, 2: unrelated sets.
__attribute__((noinline))
static void victim(int bit, int mode) {
    if (mode == 1)
        return;
    int set = mode == 0 ? bit : bit + 2;
    prime(memory + ways * stride + set * line);
}

// Touch recording pages before timing.
static void touch_pages(void *buffer, size_t bytes, size_t page) {
    volatile uint8_t *p = buffer;
    for (size_t offset = 0; offset < bytes; offset += page)
        p[offset] = 0;
    p[bytes - 1] = 0;
}

static uint64_t collect(Sample *samples, int count, int mode, int cpu) {
    // Shuffle balanced labels before timing.
    for (int i = 0; i < count; ++i) {
        samples[i].bit = i % 2;
        samples[i].order = rand() & 1;
    }
    for (int i = count - 1; i > 0; --i) {
        int j = rand() % (i + 1);
        int bit = samples[i].bit;
        samples[i].bit = samples[j].bit;
        samples[j].bit = bit;
    }
    if (sched_getcpu() != cpu) {
        fprintf(stderr, "CPU affinity check failed before batch\n");
        exit(1);
    }
    uint64_t batch_start = now_ns();
    for (int i = 0; i < count; ++i) {
        int bit = samples[i].bit;
        int order = samples[i].order;
        uint64_t start = now_ns();
        prime(memory);
        prime(memory + line);
        victim(bit, mode);
        uint64_t first = probe(memory + order * line);
        uint64_t second = probe(memory + (1 - order) * line);
        uint64_t end = now_ns();
        samples[i].a = order == 0 ? first : second;
        samples[i].b = order == 0 ? second : first;
        samples[i].ns = end - start;
    }
    uint64_t elapsed = now_ns() - batch_start;
    if (sched_getcpu() != cpu) {
        fprintf(stderr, "CPU affinity check failed after batch\n");
        exit(1);
    }
    return elapsed;
}

static int compare_scores(const void *a, const void *b) {
    int64_t x = *(const int64_t *)a, y = *(const int64_t *)b;
    return (x > y) - (x < y);
}

static double median(int64_t *scores, int count) {
    qsort(scores, (size_t)count, sizeof(*scores), compare_scores);
    return ((double)scores[(count - 1) / 2] + scores[count / 2]) / 2;
}

static void write_samples(Sample *samples, int count, const char *mode,
```

```
                          int run, double threshold) {
    const char *phase = run < 0 ? "calibration" : "evaluation";
    for (int i = 0; i < count; ++i) {
        Sample s = samples[i];
        int64_t score = (int64_t)s.b - (int64_t)s.a;
        int guess = run < 0 ? -1 : score > threshold;
        printf("%s,%s,%d,%d,%d,%d,%" PRIu64 ",%" PRIu64 ",%" PRIu64 ",%d\n",
               phase, mode, run, i, s.bit, s.order, s.a, s.b, s.ns, guess);
    }
}

static int parse_int(const char *text, int low, int high) {
    char *end;
    long n = strtol(text, &end, 10);
    if (!*text || *end || n < low || n > high) {
        fprintf(stderr, "Invalid argument: %s\n", text);
        exit(2);
    }
    return (int)n;
}

int main(int argc, char **argv) {
    if (argc != 6 && argc != 9) {
        fprintf(stderr, "Usage: %s CPU WAYS SETS LINE_BYTES SEED [TRIALS RUNS CAL_PER_BIT]\n", argv[0]);
        return 2;
    }
    int cpu = parse_int(argv[1], 0, CPU_SETSIZE - 1);
    ways = parse_int(argv[2], 1, MAX_WAYS);
    int sets = parse_int(argv[3], 4, 4096);
    line = parse_int(argv[4], 16, 1024);
    unsigned int seed = (unsigned int)parse_int(argv[5], 1, 2147483647);
    srand(seed);
    int trials = argc == 9 ? parse_int(argv[6], 2, 1000000) : 10000;
    int runs = argc == 9 ? parse_int(argv[7], 1, 100) : 10;
    int calibration = argc == 9 ? parse_int(argv[8], 2, 1000000) : 10000;
    stride = (size_t)sets * line;
    size_t page = (size_t)sysconf(_SC_PAGESIZE);
    if (trials % 2 || stride > page || (stride & (stride - 1))) {
        fprintf(stderr, "Unsupported geometry or unbalanced trial count\n");
        return 2;
    }
    unsigned int a, b, c, d;
    if (!__get_cpuid(0x80000001U, &a, &b, &c, &d) || !(d & (1U << 27))) {
        fprintf(stderr, "RDTSCP is unavailable\n");
        return 3;
    }
    cpu_set_t allowed, selected;
    if (sched_getaffinity(0, sizeof(allowed), &allowed) || !CPU_ISSET(cpu, &allowed)) {
        fprintf(stderr, "Requested CPU is not in the allowed affinity mask\n");
        return 2;
    }
    CPU_ZERO(&selected);
    CPU_SET(cpu, &selected);
    if (sched_setaffinity(0, sizeof(selected), &selected)) {
        perror("sched_setaffinity");
        return 1;
    }

    uint64_t setup_start = now_ns();
    // First half: attacker. Second half: victim.
    size_t bytes = 2 * (size_t)ways * stride;
    int capacity = trials > 2 * calibration ? trials : 2 * calibration;
    memory = aligned_alloc(page, ((bytes + page - 1) / page) * page);
    Sample *samples = calloc((size_t)capacity, sizeof(*samples));
    int64_t *scores = calloc(2 * (size_t)calibration, sizeof(*scores));
    if (!memory || !samples || !scores) {
        fprintf(stderr, "Allocation failed\n");
        free(memory);
```

```
        free(samples);
        free(scores);
        return 1;
    }
    memset(memory, 0, bytes);
    touch_pages(samples, (size_t)capacity * sizeof(*samples), page);
    touch_pages(scores, 2 * (size_t)calibration * sizeof(*scores), page);
    uint64_t setup_ns = now_ns() - setup_start;
    fprintf(stderr, "{\"type\":\"setup\",\"cpu\":%d,\"ways\":%d,\"sets\":%d,\"line_bytes\":%d,\"stride_bytes
     \":%zu,\"seed\":%u,\"setup_ns\":%" PRIu64 ",\"attack_allocation_bytes\":%zu,\"recording_allocation_bytes
     \":%zu}\n",
            cpu, ways, sets, line, stride, seed, setup_ns, bytes,
            (size_t)capacity * sizeof(*samples) + 2 * (size_t)calibration * sizeof(*scores));
    puts("phase,mode,run,trial,bit,probe_order,time_a_ticks,time_b_ticks,latency_ns,guess");

    // Calibrate once, then keep the threshold fixed.
    uint64_t calibration_ns = collect(samples, 2 * calibration, 0, cpu);
    int counts[2] = {0, 0};
    for (int i = 0; i < 2 * calibration; ++i) {
        int bit = samples[i].bit;
        scores[bit * calibration + counts[bit]++] = (int64_t)samples[i].b - (int64_t)samples[i].a;
    }
    double median0 = median(scores, calibration);
    double median1 = median(scores + calibration, calibration);
    double threshold = (median0 + median1) / 2;
    fprintf(stderr, "{\"type\":\"calibration\",\"samples_per_bit\":%d,\"median0\":%.6f,\"median1\":%.6f,\"
     threshold\":%.6f,\"elapsed_ns\":%" PRIu64 "}\n",
            calibration, median0, median1, threshold, calibration_ns);
    fflush(stderr);
    write_samples(samples, 2 * calibration, "signal", -1, threshold);

    const char *modes[] = {"signal", "no_victim", "unrelated"};
    for (int run = 0; run < runs; ++run) {
        // Rotate signal and control order.
        for (int offset = 0; offset < 3; ++offset) {
            int mode = (run + offset) % 3;
            uint64_t elapsed = collect(samples, trials, mode, cpu);
            fprintf(stderr, "{\"type\":\"batch\",\"run\":%d,\"mode\":\"%s\",\"trials\":%d,\"elapsed_ns\":%"
     PRIu64 ",\"cpu_after\":%d}\n",
                    run, modes[mode], trials, elapsed, sched_getcpu());
            write_samples(samples, trials, modes[mode], run, threshold);
        }
    }
    struct rusage usage;
    if (!getrusage(RUSAGE_SELF, &usage)) {
        fprintf(stderr, "{\"type\":\"resources\",\"max_rss_kib\":%ld,\"minor_faults\":%ld,\"major_faults\":%
     ld,\"voluntary_context_switches\":%ld,\"involuntary_context_switches\":%ld}\n",
                usage.ru_maxrss, usage.ru_minflt, usage.ru_majflt, usage.ru_nvcsw, usage.ru_nivcsw);
    }
    free(memory);
    free(samples);
    free(scores);
    return 0;
}
```

A.4 Flush+Reload (`flushreload.c`)

```
#include <stdio.h>
#include <stdint.h>
#include <stdalign.h>
#include <stdlib.h>

static uint32_t xorshift32(uint32_t *state) {
    uint32_t x = *state;
    x ^= x << 13;
    x ^= x >> 17;
```

```
    x ^= x << 5;
    return *state = x;
}

unsigned long probe (char * adrs) {
    volatile unsigned long time;
    __asm__ __volatile__ (
        " mfence \n"
        " lfence \n"
        " rdtsc \n"
        " lfence \n"
        " movl %%eax, %%esi \n"
        " movl (%1), %%eax \n"
        " lfence \n"
        " rdtsc \n"
        " subl %%esi, %%eax \n"
        " clflush 0(%1) \n"
        " mfence \n"
        : "=a" (time)
        : "c" (adrs)
        : "%esi", "%edx", "memory");
    return time;
}

int compare (const void * a, const void * b) {
    int aint = *(int*)a;
    int bint = *(int*)b;
    if (aint < bint) return -1;
    if (aint > bint) return 1;
    return 0;
}

int main(void) {
    alignas(64) static char target[64] = "test";
    int nwarm = 1000;
    unsigned long hits[1000] = {0};
    unsigned long misses[1000] = {0};
    uint32_t seed = 1234;
    uint32_t touch;
    int j = 0;
    int k = 0;

    for (int i = 0; i < nwarm; i++) {
        touch = xorshift32(&seed) % 2;
        if (touch) {
            volatile char c = *(volatile char *)target;
            (void)c;
            hits[j] = probe(target);
            j++;
        } else {
            misses[k] = probe(target);
            k++;
        }
    }

    unsigned long hit_count = 0;
    unsigned long miss_count = 0;
    for (int i = 0; i < nwarm; i++) {
        if (hits[i] != 0)   { hit_count++; }
        if (misses[i] != 0) { miss_count++; }
    }

    qsort(hits, nwarm, sizeof(unsigned long), compare);
    qsort(misses, nwarm, sizeof(unsigned long), compare);
    int hit_med  = hits[nwarm - (hit_count/2)];
    int miss_med = misses[nwarm - (miss_count/2)];
    printf("Hit Median: %lu\tMiss Median: %lu\n", hit_med, miss_med);
    printf("Hit 90 high: %lu\tMiss 10 low: %lu\n",
```

```
           hits[nwarm - (hit_count/10)], misses[nwarm - 9*(miss_count/10)]);
    unsigned long threshhold = (hits[nwarm - (hit_count/10)] +
                                misses[nwarm - 9*(miss_count/10)]) / 2;
    printf("Threshhold: %lu\n", threshhold);

    unsigned long time;
    int pred_touch;
    FILE *csv = fopen("output.csv", "w");
    FILE *txt = fopen("output.txt", "w");
    if (csv == NULL) { printf("Failed to open file output.csv\n"); }
    else             { printf("File Opened\n"); }
    fprintf(csv, "latency,predicted hit,actual hit\n");

    unsigned long times[20000];
    uint32_t touches[20000];
    for (int i = 0; i < 20000; i++) {
        touch = xorshift32(&seed) % 2;
        touches[i] = touch;
        if (touch) {
            volatile char c = *(volatile char *)target;
            (void)c;
            times[i] = probe(target);
        } else {
            times[i] = probe(target);
        }
    }

    hit_count = 0;
    miss_count = 0;
    long correct = 0;
    long wrong = 0;
    float acc = 0;
    unsigned long hit_sum = 0;
    unsigned long miss_sum = 0;
    for (int i = 0; i < 20000; i++) {
        pred_touch = times[i] < threshhold;
        if (touches[i]) { hit_count++;  hit_sum += times[i]; }
        else            { miss_count++; miss_sum += times[i]; }
        if (pred_touch == touches[i]) { correct++; }
        else                          { wrong++; }
        acc = (float)correct / ((float)wrong + (float)correct);
        fprintf(csv, "%lu,%d,%d\n", times[i], pred_touch, touches[i]);
    }

    fprintf(txt, "Average hit: %d\tAverage miss: %d\n",
            hit_sum/hit_count, miss_sum/miss_count);
    fprintf(txt, "Correct: %d\tWrong: %d\tAccuracy: %lf", correct, wrong, acc);
    fclose(csv);
    fclose(txt);
    return 0;
}
```

B DMP Detection Code

B.1 dmp_find.c

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <x86intrin.h>
#include "find_threshold.h"

#define PAGE_SIZE 4096
```

```
#define LINE_SIZE 64
#define POOL_PAGES 1024
#define ARRAY_ENTRIES 64
#define STREAMED_ENTRIES 32
#define MAX_DIST (ARRAY_ENTRIES - STREAMED_ENTRIES)
#define TRIALS 20000
#define COLD_SAMPLES 4000

// pointer + garbage is dmp test and sw prefetch are controls that stay cached
enum dmp_condition { MODE_POINTER, MODE_GARBAGE, MODE_SW_PREFETCH, MODE_SW_PREFETCH_T2 };

static uint8_t target_pool[POOL_PAGES * PAGE_SIZE] __attribute__((aligned(4096)));
static uint64_t ptr_array[ARRAY_ENTRIES] __attribute__((aligned(4096)));

// offsets so cant do dmp directly
static uint32_t target_offsets[ARRAY_ENTRIES];

// xorshift to make random numbers from stackoverflow
static uint32_t rng_state;
static uint32_t next_rand(void) {
    uint32_t x = rng_state;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return rng_state = x;
}

static volatile uint64_t sink; // keeps the stream sum from being optimized out

// for qsort smallest first
static int cmp_cycles(const void *first_ptr, const void *second_ptr) {
    uint64_t first  = *(const uint64_t *)first_ptr;
    uint64_t second = *(const uint64_t *)second_ptr;
    return (first > second) - (first < second);
}

// time flushed reads of random lines to sanity check calibration
static void check_cold_miss(void) {
    static uint64_t cold_samples[COLD_SAMPLES];
    rng_state = 88675123u;
    for (int i = 0; i < COLD_SAMPLES; i++) {
        uint32_t page = next_rand() % POOL_PAGES;
        uint32_t line = next_rand() % (PAGE_SIZE / LINE_SIZE);
        uint8_t *rand_line = &target_pool[page * PAGE_SIZE + line * LINE_SIZE];
        _mm_clflush(rand_line);
        _mm_mfence();
        cold_samples[i] = access_time(rand_line);
    }
    qsort(cold_samples, COLD_SAMPLES, sizeof(uint64_t), cmp_cycles);
    printf("random-line miss median/5p : %llu / %llu cyc\n",
           (unsigned long long)cold_samples[COLD_SAMPLES / 2],
           (unsigned long long)cold_samples[(int)(5.0 / 100.0 * (COLD_SAMPLES - 1))]);
}

// runs one mode and fills per-distance hit rates then returns overall hit rate
static double run(const char *label, enum dmp_condition mode, double *rate_by_dist) {
    int hits_by_dist[MAX_DIST + 1] = {0};
    int trials_by_dist[MAX_DIST + 1] = {0};
    int total_hits = 0;
    rng_state = 2463534242u;

    for (int trial = 0; trial < TRIALS; trial++) {
        // new random page and line for every target
        for (int entry = 0; entry < ARRAY_ENTRIES; entry++) {
            uint32_t page = next_rand() % POOL_PAGES;
            uint32_t line = next_rand() % (PAGE_SIZE / LINE_SIZE);
            target_offsets[entry] = page * PAGE_SIZE + line * LINE_SIZE;
            ptr_array[entry] = (uint64_t)(uintptr_t)&target_pool[target_offsets[entry]];
```

```
        }

        // probe one entry past the streamed half; distance 1 = first unread entry
        int dist = 1 + next_rand() % MAX_DIST;
        int probed_entry = STREAMED_ENTRIES - 1 + dist;
        uint8_t *probed_target = &target_pool[target_offsets[probed_entry]];

        // garbage control so only the probed entry stops looking like a pointer
        if (mode == MODE_GARBAGE)
            ptr_array[probed_entry] = 0xdead000000000000ull | (uint64_t)trial;

        // flush every target and the whole pointer array
        _mm_mfence();
        for (int entry = 0; entry < ARRAY_ENTRIES; entry++)
            _mm_clflush(&target_pool[target_offsets[entry]]);
        for (int line = 0; line < ARRAY_ENTRIES * 8 / LINE_SIZE; line++)
            _mm_clflush(&ptr_array[line * LINE_SIZE / 8]);
        _mm_mfence();

        // read the first half as plain numbers and keep referenced
        volatile uint64_t *stream = ptr_array;
        uint64_t sum = 0;
        for (int entry = 0; entry < STREAMED_ENTRIES; entry++)
            sum += stream[entry];
        sink = sum;

        // prefetch into L1, make sure it can see it
        if (mode == MODE_SW_PREFETCH)
            _mm_prefetch((const char *)probed_target, _MM_HINT_T0);
        // prefetch into outer caches now
        if (mode == MODE_SW_PREFETCH_T2)
            _mm_prefetch((const char *)probed_target, _MM_HINT_T2);

        for (volatile int delay = 0; delay < 1000; delay++) {} // give any prefetch time to land

        trials_by_dist[dist]++;
        if (access_time(probed_target) < threshold) {
            hits_by_dist[dist]++;
            total_hits++;
        }
    }

    for (int dist = 1; dist <= MAX_DIST; dist++)
        rate_by_dist[dist] = trials_by_dist[dist]
            ? 100.0 * hits_by_dist[dist] / trials_by_dist[dist]
            : 0.0;

    double rate = 100.0 * total_hits / TRIALS;
    printf("%-12s cached-after-flush: %6.2f%%\n", label, rate);
    return rate;
}

int main(void) {
    static double prefetch_by_dist[MAX_DIST + 1];
    static double prefetch_t2_by_dist[MAX_DIST + 1];
    static double pointer_by_dist[MAX_DIST + 1];
    static double garbage_by_dist[MAX_DIST + 1];

    // output to file
    if (freopen("dmp_results.txt", "w", stdout) == NULL) {
        perror("dmp_results.txt");
        return 1;
    }

    memset(target_pool, 1, sizeof target_pool); // give every page real memory
    if (calibrate() != 0)
        printf("error with calibration\n");
    check_cold_miss();
```

```
    double s = run("sw-prefetch",    MODE_SW_PREFETCH,    prefetch_by_dist);
    run("sw-prefetch-t2", MODE_SW_PREFETCH_T2, prefetch_t2_by_dist);
    double p = run("pointerlike", MODE_POINTER, pointer_by_dist);
    double g = run("garbage",     MODE_GARBAGE, garbage_by_dist);

    // track gap, useful for debug
    printf("\ndistance pointerlike garbage\n");
    double best_delta = -100.0;
    int best_dist = 1;
    for (int dist = 1; dist <= MAX_DIST; dist++) {
        printf("%8d %10.2f%% %6.2f%%\n", dist, pointer_by_dist[dist], garbage_by_dist[dist]);
        double delta = pointer_by_dist[dist] - garbage_by_dist[dist];
        if (delta > best_delta) { best_delta = delta; best_dist = dist; }
    }
    printf("\nDelta (pointerlike - garbage): %.2f%%\n", p - g);
    printf("Best distance %d delta: %.2f%%\n", best_dist, best_delta);

    // check controls for baseline then judge real numbers on the best distance
    if (s < 90.0)
        puts("=> invalid, probe can't see a software prefetch");
    else if (g > 10.0)
        puts("=> invalid baseline too high");
    else
        puts(best_delta > 10.0
             ? "=> DMP behavior likely due to cache behavior"
             : "=> DMP unlikely due to cache behavior..");
    return 0;
}
```

B.2 find_threshold.h

```
#ifndef FIND_THRESHOLD_H
#define FIND_THRESHOLD_H

#include <stdint.h>

extern uint64_t threshold;  // below this is cache hit; gets set by calibrate()
extern uint64_t timer_ovh;  // cycles the timing code costs by itself

int calibrate(void);                        // sets threshold; returns -1 if hits and misses overlap
uint64_t access_time(volatile uint8_t *p);  // cycles to read 1 byte minus overhead

#endif
```

B.3 find_threshold.c

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <x86intrin.h>
#include "find_threshold.h"

#define SAMPLES 4000
#define MISS_BUF_SIZE (4 * 1024 * 1024)
#define MISS_LINES (MISS_BUF_SIZE / 64)

uint64_t threshold = 0;  // below this = cache hit
uint64_t timer_ovh = 0;  // cycles the timing code costs by itself

// one cache line on its own page so nothing else shares it
static uint8_t hit_line[64] __attribute__((aligned(4096)));
```

```
// misses come from random lines in here, same as the real probes
static uint8_t miss_buf[MISS_BUF_SIZE] __attribute__((aligned(4096)));

// for qsort, smallest first
static int cmp_cycles(const void *first_ptr, const void *second_ptr) {
    uint64_t first  = *(const uint64_t *)first_ptr;
    uint64_t second = *(const uint64_t *)second_ptr;
    return (first > second) - (first < second);
}

// cycles to read one byte, minus timer overhead
uint64_t access_time(volatile uint8_t *addr) {
    unsigned int core_id;  // rdtscp needs this
    _mm_mfence();          // let earlier loads/flushes finish
    uint64_t start = __rdtscp(&core_id);
    (void)*addr;
    _mm_lfence();          // wait for the load before stopping the timer
    uint64_t end = __rdtscp(&core_id);
    uint64_t elapsed = end - start;
    // clamp to 0 for unsigned op
    if (elapsed > timer_ovh)
        return elapsed - timer_ovh;
    return 0;
}

int calibrate(void) {
    static uint64_t ovh_samples[SAMPLES];
    static uint64_t hit_samples[SAMPLES];
    static uint64_t miss_samples[SAMPLES];
    volatile uint8_t *hit_addr = hit_line;
    unsigned int core_id;

    // timer overhead for same fences
    timer_ovh = 0;
    for (int i = 0; i < SAMPLES; i++) {
        _mm_mfence();
        uint64_t start = __rdtscp(&core_id);
        _mm_lfence();
        uint64_t end = __rdtscp(&core_id);
        ovh_samples[i] = end - start;
    }
    qsort(ovh_samples, SAMPLES, sizeof(uint64_t), cmp_cycles);
    timer_ovh = ovh_samples[SAMPLES / 2]; // median

    // hits: touch line right before timing
    (void)*hit_addr;
    for (int i = 0; i < SAMPLES; i++) {
        (void)*hit_addr;
        hit_samples[i] = access_time(hit_addr);
    }

    // misses: flush random line right before timing
    // added this in because reusing one line gave miss times that were all over the place
    memset(miss_buf, 1, sizeof miss_buf); // give every page real memory
    uint32_t rng = 2463534242u;
    for (int i = 0; i < SAMPLES; i++) {
        rng ^= rng << 13;
        rng ^= rng >> 17;
        rng ^= rng << 5;
        uint32_t line_idx = rng % MISS_LINES;
        volatile uint8_t *miss_addr = &miss_buf[line_idx * 64];
        _mm_clflush((void *)miss_addr);
        _mm_mfence();
        miss_samples[i] = access_time(miss_addr);
    }

    qsort(hit_samples, SAMPLES, sizeof(uint64_t), cmp_cycles);
    qsort(miss_samples, SAMPLES, sizeof(uint64_t), cmp_cycles);
```

```
    int p95_idx = (int)(95.0 / 100.0 * (SAMPLES - 1));
    int p5_idx  = (int)(5.0  / 100.0 * (SAMPLES - 1));
    uint64_t hit_median  = hit_samples[SAMPLES / 2];
    uint64_t miss_median = miss_samples[SAMPLES / 2];
    uint64_t hit_p95 = hit_samples[p95_idx];  // slow end of hits
    uint64_t miss_p5 = miss_samples[p5_idx];  // fast end of misses

    printf("timer overhead : %llu cyc\n", (unsigned long long)timer_ovh);
    printf("hit median/95p : %llu / %llu cyc\n",
           (unsigned long long)hit_median, (unsigned long long)hit_p95);
    printf("miss median/5p : %llu / %llu cyc\n",
           (unsigned long long)miss_median, (unsigned long long)miss_p5);

    // hits and misses were overlapping so fall back to midpoint of medians
    if (miss_p5 <= hit_p95) {
        printf("error with test, retry (miss_5p=%llu <= hit_95p=%llu). retry.\n",
               (unsigned long long)miss_p5, (unsigned long long)hit_p95);
        threshold = (hit_median + miss_median) / 2;
        return -1;
    }

    // go to midgap of hits + misses
    threshold = (hit_p95 + miss_p5) / 2;
    printf("=> threshold set to %llu cyc (between hit_95p and miss_5p)\n",
           (unsigned long long)threshold);
    return 0;
}
```